



Ministère de l'Enseignement Supérieur et de la recherche scientifique

Direction Générale des Etudes Technologiques

Institut supérieur des Etudes Technologiques de Sfax

Département technologie de l'informatique



Code Stage : DSI3/44/22

Soutenu le : **/06/2023

STAGE DE FIN D'ETUDES

Pour l'obtention du diplôme

LICENCE APPLIQUEE EN TECHNOLOGIES DE L'INFORMATIQUE

Spécialité : DEVELOPPEMENT DES SYSTEMES D'INFORMATIONS (DSI)

Développement d'une application de messageries

ORGANISME D'ACCUEIL : Data Sphera

ELABORE PAR : Kharrat Sana & mkaouar Eya

JURY :

Président : Mallouli Oussema

Rapporteur : Elleuch Mohamed

Encadreur : Jemal Ahmed

Année Universitaire : 2022/2023

Dédicaces

Avec l'expression de toute notre reconnaissance, nous dédions ce travail :

À nous-mêmes

Pour nos sacrifices et notre travail dur tout au long de notre projet de fin d'études.

À nos chers parents

Nos sources de tendresse et d'amour pour leur soutien tout au long de notre vie scolaire et universitaire, et pour tout ce qu'ils ont fait pour nous aider.

À nos frères et sœurs

Pour les beaux moments que nous avons passés ensemble et que nous chérissons.

À tous ceux qui ont contribué

Que ce travail soit l'expression de notre grande affectation et un témoignage de notre attachement et de notre profond amour. Nous prions que dieu leur accorde santé et prospérité.

Remerciements

Au terme de ce travail, nous tenons à exprimer nos sincères gratitude à tous ceux qui ont pensé à la réalisation de ce projet de fin d'études. Vos encouragements, votre soutien et votre expertise ont été essentiels à notre réussite.

Nous adressons nos sincères remerciements à notre encadreur de l'ISET, M. Ahmed Jmal, pour ses efforts pour nous guider, pour son soutien et ses conseils avisés tout au long de ce travail. Nous lui témoignons un grand respect pour ses qualités humaines et professionnelles.

Nous tenons également à remercier nos encadreur de la société, M. Noureddine Loukil, M. Ahmed Jarraya et M. Nizar Arous pour leur accueil. Grâce à leur patience et à leur expertise, nous avons pu surmonter les obstacles et atteindre nos objectifs.

Nous tenons à remercier tous les membres de jury qui nous font l'honneur d'évaluer ce modeste travail.

Enfin, nous ferons à exprimer notre reconnaissance à toutes les personnes qui nous ont encouragées et soutenues pendant toute cette période. Sans vous, nous n'aurions pas réussi à atteindre nos objectifs. Nous vous sommes infiniment reconnaissants.

Table des matières

Introduction Générale.....	1
Etude préalable.....	2
1. Introduction	3
2. Présentation de l'organisme d'accueil.....	3
2.1 Présentation de l'entreprise.....	3
2.2 Domaines de compétences.....	3
2.3 Points forts	4
3. Contexte du projet	4
3.1 Présentation du projet.....	4
3.2 Etude de l'existant	5
3.3 Critique de l'existant.....	5
3.4 Problématique.....	6
3.5 Les objectifs.....	6
3.6 Captures des besoins	6
4. Méthodologie de conception	8
4.1 Langage de modélisation.....	8
4.2 Méthodologie de conception	9
5. Backlog Produit.....	11
6. Conclusion.....	13
Sprint0 : Architecture Logicielle et Fonctionnelle.....	14
1. Introduction	15
2. Backlog Sprint 0.....	15
3. Vue architecturale du système.....	15
3.1 Présentation d'Exacctlly	16
3.2 Présentation de l'interface du Widget	18
3.3Présentation Socket.IO.....	19
4. Conception globale.....	20
4.1 Diagramme de cas d'utilisation global.....	20
4.2 Diagramme de classes.....	21
4.3 Diagramme de séquence.....	22
4.4 Diagramme de déploiement	24

5.	Fonctionnement.....	25
6.	Installation de l'environnement du projet.....	25
6.1	Clonage du projet	26
6.2	Build de projet	26
6.3	Résultat de l'exécution	28
7.	Outils et environnement logiciel	28
7.1	Environnement de développement intégré.....	28
7.2	Système de Gestion de Base de données	28
7.3	Framework.....	29
7.4	Langages de programmation.....	29
7.5	Outils de test.....	30
8.	Conclusion.....	30
	Sprint 1 et 2:	31
	Connexion et Collecter les données des visiteurs	31
	Téléchargement d'une image ou d'un fichier	31
1.	Introduction	32
	Partie 1 : Sprint 1	32
2.	Backlog du Sprint.....	32
3.	Analyse et spécification des besoins	32
3.1	Description	32
3.2	Diagramme des cas d'utilisations	32
4.	Maquettage	35
5.	Conception.....	37
5.1	Diagramme de classes.....	37
5.2	Diagramme de séquence.....	38
6.	Réalisation	38
	Partie 2 : Sprint 2	42
1.	Backlog Sprint 2.....	42
2.	Analyse et spécification des besoins	42
2.1	Description	42
2.2	Diagramme de cas d'utilisation	42
3.	Schéma explicatif.....	44
4.	Maquettage	45
4.	Conception.....	45
4.1	Diagramme de classes.....	45
4.2	Diagramme de séquence.....	46

5. Réalisation	46
Explication du Redux	Erreur ! Signet non défini.
6. Conclusion.....	48
Sprint 3, 4 et 5 :	49
Interface des questions fréquemment posées	49
Interface des nouvelles	49
Création d'interface accueil	49
1. Introduction	50
Partie 1 : Sprint 3	50
2. Backlog du Sprint.....	50
3. Analyse et spécification des besoins	50
3.1 Description	50
3.2 Diagramme de cas d'utilisation	50
4. Maquettage	51
5. Conception.....	52
5.1 Diagramme de classes	52
5.2 Diagramme de séquence	52
6. Réalisation	52
Partie 2 : Sprint 4	55
1. Backlog du Sprint.....	55
2. Diagramme de cas d'utilisation.....	55
2.1 Diagramme de cas d'utilisation	55
2.2 Description textuelle du cas d'utilisation	56
3. Maquettage :	56
4. Conception.....	57
4.1 Diagramme de classes:	57
4.2 Diagramme de séquence :	57
5. Réalisation :	57
Partie 3 : Sprint 5	60
1. Backlog du Sprint.....	60
2. Diagramme de cas d'utilisation.....	60
2.1 Diagramme de cas d'utilisation	60
2.2 Description textuelle du cas d'utilisation	61
3. Maquettage :	61
4. Conception.....	62
4.1 Diagramme de classes	62

4.2	Diagramme de séquence.....	63
5.	Réalisation	63
	Conclusion générale et perspectives.....	65
	Néographie	66

Liste des figures

Figure 1: logo DataSphera	3
Figure 2:logo Exacctly	3
Figure 3:webchat de Rasa	4
Figure 4:logo de rasa	5
Figure 5: modélisation d'un système	8
Figure 6 : logo de Draw.io	8
Figure 7:la méthodologie scrum.....	10
Figure 8:architecture multiniveau	16
Figure 9:Interface des intégrations d'exactly	17
Figure 10: Interface de personnalisation du web chat.....	17
Figure 11:Interface de conversations.....	18
Figure 12: Interface widget	19
Figure 13: Socket.io	19
Figure 14:diagramme de cas d'utilisation global.....	20
Figure 15:diagramme de classe global	21
Figure 16:diagramme de séquence global	23
Figure 17:diagramme de déploiement.....	24
Figure 18:diagramme de déploiement global	25
Figure 19:fonctionnement de projet.....	25
Figure 20:Etapes d'installation projet	25
Figure 21: le lien de projet dans gitlab	26
Figure 22:la commande de clonage dans visual studio code	26
Figure 23:schéma explicatif du command de build.....	27
Figure 24:la commande d'installation npm 6.....	27
Figure 25: installation de dépendances	27
Figure 26:web chat.....	28
Figure 27:visual studio code.....	28
Figure 28:firebase.....	28
Figure 29:mongoDB.....	29
Figure 30:reactJs.....	29
Figure 31:nodejs	29
Figure 32:Redux.....	29
Figure 33:html 5	29
Figure 34:css	29
Figure 35:java script	30
Figure 36:spring boot	30
Figure 37:Postman.....	30
Figure 38:diagramme de cas d'utilisation en cas invité	33
Figure 39:diagramme de cas d'utilisation en cas visiteur	33
Figure 40:maquette interface messages.....	35
Figure 41:maquette d'interface de connexion.....	36
Figure 42:Diagramme de classes de la connexion.....	37
Figure 43:diagramme de classes	37

Figure 44:diagramme de séquence sprint 1	38
Figure 45:interface messages.....	39
Figure 46:la méthode envGuest	39
Figure 47:la réception de données de méthode "envGuest" coté exacttly.....	40
Figure 48:méthode envoyer	40
Figure 49:interface de connexion.....	41
Figure 50:la réception de données de méthode "env" coté exacttly	41
Figure 51:diagramme de cas d'utilisation sprint 2	42
Figure 52:schéma explicatif.....	44
Figure 53:maquette de l'interface de bouton de téléchargement	45
Figure 54:diagramme de classes sprint2	45
Figure 55:diagramme de séquence de sprint 2.....	46
Figure 56:création de projet firebase.....	46
Figure 57:configuration de firebase	47
Figure 58:code de bouton télécharger.....	47
Figure 59:interface webchat	48
Figure 60:diagramme de cas d'utilisation sprint 3	50
Figure 61: logo FAQ	51
Figure 62: diagramme de classes sprint 3	52
Figure 63:diag de séquence de faq.....	52
Figure 64:récupération de token de site exacttly	53
Figure 65:test de l'api avec postman.....	53
Figure 66:code faq	54
Figure 67:code de récupération 'api d'exacttly	54
Figure 68:structure de la classe « nouvelles »	58
Figure 69:la table "nouvelles " dans Mongo DB.....	58
Figure 70:interface nouvelles	59
Figure 71: clique sur un nouvelle	59
Figure 72:diagramme de cas d'utilisation sprint 5	60
Figure 73:maquette de l'interface d'accueil.....	61
Figure 74:diagramme de classes sprint 5	62
Figure 75:diagramme de séquence sprint 5	63
Figure 76:installation les packages de mui material	63
Figure 77:les importations de package mui	63
Figure 78:code de menu.....	64
Figure 79:interface d'accueil	64

Liste des tableaux

Tableau 1: besoins fonctionnels	7
Tableau 2: Besoins non fonctionnels	7
Tableau 3: backlog produit	12
Tableau 4: Découpage en sprints	13
Tableau 5: sprint 0	15
Tableau 6: explication de socket	20
Tableau 7: Description des composants du diagramme de classe	22
Tableau 8: backlog sprint 1	32
Tableau 9: description textuelle en mode invité	33
Tableau 10: description textuelle en mode visiteur	34
Tableau 11: Backlog sprint 2	42
Tableau 12: description de cas d'utilisation sprint 2	43
Tableau 13: backlog sprint 3	50
Tableau 14: description textuelle du cas d'utilisation sprint 3	51
Tableau 15: backlog sprint 4	55
Tableau 16: description de cas d'utilisation sprint 4	56
Tableau 17: description textuelle de diagramme de cas d'utilisation sprint 5	61

Introduction Générale

Le projet de fin d'études est une opportunité pour mettre en valeur nos compétences et connaissances en informatique, acquises grâce à notre formation universitaire à l'Institut Supérieur des Études Technologiques de Sfax.

Dans le contexte de la croissance exponentielle des applications web, les fonctionnalités de messagerie intégrées sont devenues essentielles pour les entreprises et organisations, facilitant les échanges d'informations, les discussions de groupe et la planification des tâches.

Cependant, la gestion de ces web chats peut être complexe, d'où l'utilisation de l'intelligence artificielle pour améliorer l'expérience utilisateur. Alors, dans le cadre de notre projet, nous avons pour objectif d'améliorer un web chat basé sur Rasa.

Afin d'illustrer la démarche de notre travail, nous vous présentons dans ce qui suit l'organisation générale du ce présent rapport qui s'articule autour de 4 chapitres.

Le premier chapitre intitulé « Etude préalable » présente le contexte général de projet, l'étude de l'existant, les besoins fonctionnels et non fonctionnelle, le pilotage de notre projet avec la méthodologie Scrum ainsi le backlog produit.

Le deuxième chapitre intitulé « Sprint0 : Architecture Logicielle et Fonctionnelle » présente l'architecture de notre projet, les installations nécessaires pour configurer notre projet, la conception générale, ainsi les outils des environnements utilisés.

Les deux chapitres suivants intitulé « Sprint1 et 2 : Connexion et Collecter les données des visiteurs , Téléchargement d'une image ou d'un fichier » et « Sprint 3, 4 et 5 : Interface des questions fréquemment posées , Interface des nouvelles , Création de l'interface d'accueil » se concentreront sur l'étude dans laquelle nous définirons l'objectif, le Backlog Sprint et nous présenterons une analyse et une spécification du sprint en précisant le diagramme de cas d'utilisation et les maquetages. De plus, nous effectuerons la conception en déterminant les diagrammes de séquence et de classes. Finalement, nous achèverons la réalisation.

Chapitre I :

Etude préalable

1. Introduction

Dans ce chapitre, nous visons à replacer notre projet de fin d'études dans son contexte général.

Au premier temps, nous présentons l'organisme d'accueil DATA SPHERA, ses objectifs et ses différents domaines de compétences.

Par la suite, Nous décrivons notre projet en mettant en évidence l'étude et le critique de l'existant, le problématique à résoudre et les objectifs à atteindre.

Et finalement, nous abordons la méthodologie adoptée et les besoins fonctionnels et non fonctionnels.

2. Présentation de l'organisme d'accueil

2.1 Présentation de l'entreprise

DataSphera [N1] est une entreprise de services et de conseil informatiques spécialisée dans les solutions de support client, avec un siège social à Sfax et à Tunisie. Elle a été fondée en 2020 et compte actuellement de 4 à 10 employés.



Figure 1: logo DataSphera

DataSphera se concentre sur le développement de la plateforme "Exacctly", qui est basée sur le cloud et alimentée par l'Intelligence Artificielle (IA). Cette plateforme vise à aider les équipes de support client à augmenter leur productivité et à offrir une expérience client de qualité supérieure en utilisant des algorithmes de traitement du langage naturel (NLP) et de l'IA pour classer et router les tickets de support vers les bons agents. Exacctly utilise également une base de connaissances de l'équipe et un journal des tickets pour suggérer les réponses les plus appropriées aux agents, ce qui facilite leur travail et améliore la qualité du service client.



Figure 2: logo Exacctly

2.2 Domaines de compétences

Data sphera fournit des fonctionnalités intelligentes pour un meilleur service client:

- Réponse instantanée : Répondez à vos clients en quelques secondes avec un chatbot puissant.
- Affectation rapide : Attribue automatiquement chaque ticket au bon agent.

- Analyse des sentiments : Le suivi de la satisfaction basé sur l'IA fonctionne même sans questionnaire.
- Marquage automatique : Marquez automatiquement vos conversations en fonction du contexte.
- Support Chatbot: Activez votre chatbot pour répondre à vos clients et assister vos agents.
- Rendement de l'équipe : Suivez les indicateurs les plus importantes et détectez les obstacles de performances.

2.3 Points forts

- Amélioration continue : L'objectif est de toujours apprendre, grandir et se développer au niveau individuel et organisationnel.
- Excellence : Nous visons l'excellence et une qualité supérieure dans tous les aspects de notre entreprise.
- Orientation client : Nous plaçons les besoins de nos clients au premier plan de toutes nos décisions et actions.

3. Contexte du projet

Dans le cadre de notre formation à l'ISSET, nous sommes amenés à clôturer nos études par un stage de fin d'études. Ce stage permet de consolider les connaissances acquises tout au long de notre parcours de formation. Dans ce qui suit, nous présentons notre stage de fin d'études, en effet, il s'intéresse.

3.1 Présentation du projet

Notre Mission dans le cadre de ce projet consiste à améliorer une application de messagerie déjà réalisée avec Rasa. Alors, nous sommes en mesure d'identifier efficacement l'étude de l'existant, les problèmes et les objectifs pour nos utilisateurs.

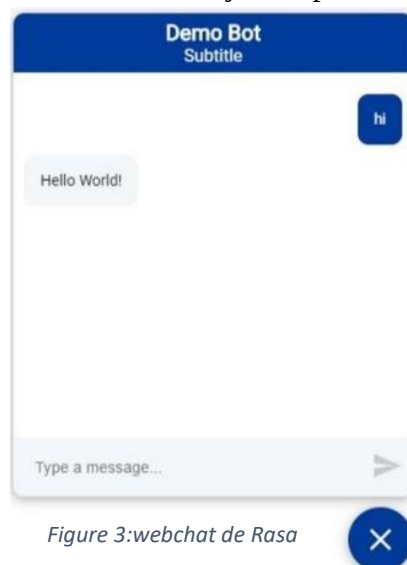


Figure 3:webchat de Rasa

Présentation de Rasa

Rasa [N2] est une plate-forme d'IA conversationnelle open source. Rasa a été créé en 2016 en tant que projet sur Git Hub et a depuis déménagé de Berlin à San Francisco.

Sa mission est de fournir une boîte à outils de bot-building basée sur l'IA qui permet de comprendre et de tenir des conversations, ainsi que de se connecter à des canaux de messagerie et à des systèmes tiers via un ensemble d'API.



Figure 4: logo de rasa

3.2 Etude de l'existant

Les clients peuvent communiquer directement avec l'équipe d'Exacctly via un chatbot, basé sur la plateforme Rasa gérer efficacement les interactions des utilisateurs. Ce chatbot permet aux utilisateurs d'envoyer des messages textuels ou des émojis, pour une communication fluide et facile.

Cependant, ce chatbot présente certaines limitations en termes de fonctionnalités importantes, que nous souhaitons clarifier ci-dessous :

- Le manque d'informations personnelles sur les clients peut retarder la réponse de l'administrateur d'Exacctly pour savoir vu qu'il ne sait pas l'identité du client qui a envoyé le message.
- Le fait de ne pas envoyer d'images limite la capacité du chatbot à fournir des informations précises et pertinentes aux visiteurs.
- Le manque de FAQ's pour les requêtes répétées peut frustrer les visiteurs qui cherchent des réponses aux questions fréquemment posées.
- Le manque d'une section des nouvelles même dans l'application exacctly réduit l'intérêt et l'engagement des utilisateurs, ainsi que la crédibilité de l'outil.

Ainsi la version standard du chatbot manque d'appropriation du contenu par l'utilisateur.

3.3 Critique de l'existant

Il existe certaines limites à la gestion des interactions des clients avec le chatbot Rasa. Premièrement, le chatbot est relativement simple et manque de fonctionnalités importantes

telles que la personnalisation, la création de FAQ, le téléchargement d'image et la section des nouvelles. De plus, le chatbot ne fournit pas d'informations personnelles sur chaque client, ce qui limite la capacité de l'entreprise à suivre les interactions des clients et à améliorer leur expérience. Ces lacunes peuvent nuire à la qualité du service fourni aux clients.

3.4 Problématique

Alors, comment apporter plus de personnalisation et de fonctionnalités conviviales aux chatbots standard.

3.5 Les objectifs

Dans le cadre de ce projet, l'objectif principal est d'améliorer les fonctionnalités du chatbot Rasa, La société Data Sphera souhaite développer une application dont les principaux objectifs sont les suivants :

- Améliorer l'expérience utilisateur en proposant des réponses pertinentes et en augmentant l'efficacité du traitement des demandes.
- Garantir une gestion efficace des interactions avec les clients en développant des fonctionnalités telles que :

-La personnalisation.

-La création d'un FAQ's.

-Le téléchargement des images et des fichiers.

-La création d'une section de nouvelles.

-La collection des informations personnelles des clients.

3.6 Captures des besoins

L'analyse du sujet nous a permis de dégager les diverses fonctionnalités que l'application devrait satisfaire.

3.6.1 Présentation des acteurs

L'acteur principal dans ce système est l'utilisateur. Il s'agit d'une personne qui accède à l'application web chat et interagit avec ses différentes fonctionnalités. L'utilisateur peut être un visiteur, un invité occasionnel, un client existant ou un employé de l'entreprise.

3.6.2 Besoins fonctionnels

Il s'agit des fonctionnalités du système. Ce sont les besoins spécifiant un comportement d'entrée/sortie du système.

Acteurs	Rôle
Utilisateur	• L'utilisateur peut consulter la page d'accueil. • L'utilisateur peut télécharger une image ou un fichier. • L'utilisateur peut accéder directement au widget ou après avoir entré leur nom et leur email. • L'utilisateur peut consulter la page de FAQ's. • L'utilisateur peut consulter la page des nouvelles.

Tableau 1: besoins fonctionnels

3.6.3 Besoins non fonctionnels

Les besoins non fonctionnels sont des exigences qui ne concernent pas spécifiquement le comportement du système mais identifient plutôt les contraintes du système. Parmi ces besoins, nous citons :

Critère	Définition
La maintenabilité	Le code doit être modulaire, commenté et doit respecter les règles standards et universelles de codage.
La performance	L'application doit être fiable, rapide et pratique.
L'ergonomie et souplesse	L'application doit offrir une interface qui respecte les critères d'ergonomie, cette interface doit également être exploitable et interactive.
Rapidité	L'application doit optimiser le traitement pour avoir un temps de réponse réduit.
L'évolutivité	Le code doit être extensible et capable d'accueillir les modifications susceptibles d'être ajoutés selon les besoins.

Tableau 2: Besoins non fonctionnels

4. Méthodologie de conception

Dans cette partie, nous définissons le langage de modélisation UML. Par la suite nous présenterons la méthodologie adoptée.

4.1 Langage de modélisation

Un langage de modélisation est un langage artificiel qui peut être utilisé pour exprimer de l'information ou de la connaissance ou des systèmes dans une structure qui est définie par un ensemble cohérent de règles.

4.1.1 Langage de modélisation choisi : UML

UML (Unified Modeling Language), que l'on peut traduire par « langage de modélisation unifié » est une notation permettant de modéliser un problème de façon standard.

4.1.2 Axes de modélisation

UML modélise le système suivant trois modes de représentation : le premier décrit les services fonctionnels rendus par le système, le second concerne la structure statique du système et le troisième concerne sa dynamique de fonctionnement.

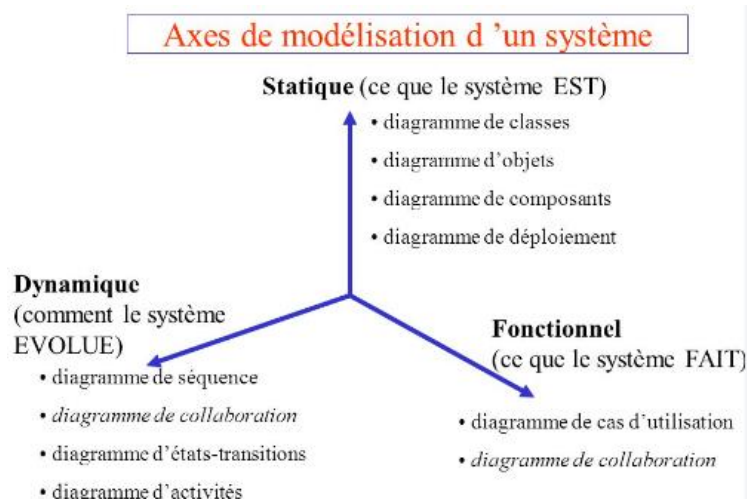


Figure 5: modélisation d'un système

4.1.3 Outil de modélisation UML

Pour la modélisation UML, nous avons choisi « Draw.io » comme outil.

Diagrams.net (draw.io) est un logiciel de dessin graphique multiplateforme gratuit et open source développé en HTML5 et JavaScript. Son interface peut être utilisée pour créer des diagrammes tels que des organigrammes, des structures filaires, des diagrammes UML, des organigrammes et des diagrammes de réseau.



Figure 6 : logo de Draw.io

4.2 Méthodologie de conception

Afin de modéliser et concevoir notre projet, notre choix s'est orienté vers UML comme langage de modélisation.

Le langage de modélisation graphique UML est un outil normalisé qui utilise des pictogrammes pour visualiser les concepts liés au développement logiciel et à la conception orientée objet. Son objectif est de simplifier la création de documents nécessaires au développement de logiciels orientés objet en étant un standard de modélisation pour l'architecture logicielle.

4.2.1 Méthode « Scrum »

Les méthodes dites « agiles » ont trouvé leurs places au sein des entreprises, et principalement des startups. Cela permet une gestion réactive et incrémentale des différents projets de l'entreprise. La plus connue de ces méthodes est très certainement la méthode « SCRUM ». Pour bien conduire notre projet et nous assurer du bon déroulement des différentes phases, nous avons opté pour SCRUM comme méthodologie de conception et de développement.

SCRUM est un processus empirique : il se base sur l'expérience du terrain. Il s'appuie sur trois piliers : la transparence, l'inspection et l'adaptation. Il suit également les principes de la culture agile.

- Transparence : SCRUM met l'accent sur le fait d'avoir un langage commun entre l'équipe et le management. Ce langage commun doit permettre à tout observateur d'obtenir rapidement une bonne compréhension du projet.

- Inspection : À intervalle régulier, SCRUM propose de faire le point sur les différents artefacts produits, afin de détecter toute variation indésirable. Ces inspections ne doivent pas être faites trop fréquemment, ou par un inspecteur mal formé : cela nuirait à l'avancement du projet.

- Adaptation : Si une dérive est constatée pendant l'inspection, le processus doit alors être adapté. SCRUM fournit des événements, durant lesquels cette adaptation est possible. Il s'agit de la réunion de planification de sprint, de la mêlée quotidienne, de la revue de sprint ainsi que de la rétrospective de sprint.

4.2.1.1 Principes et phases

SCRUM est issu des travaux de deux des signataires du manifeste Agile2, Ken Schwaber et Jeff Sutherland, au début des années 1990. Il appartient à la famille des méthodologies itératives incrémentales et repose sur les principes et les valeurs agiles qui sont quatre valeurs fondamentales :

- Les Individus et leurs interactions plus que les processus et les outils
- Des logiciels opérationnels plus qu'une documentation exhaustive

- La collaboration avec les clients plus que la négociation contractuelle
- L'adaptation au changement plus que le suivi d'un plan

Le plus souvent, les experts de SCRUM, même ses fondateurs, le décrivent comme un cadre ou un patron de processus orienté gestion de projet et qui incorpore différentes méthodes ou pratiques d'ingénierie. S'il est difficile de définir la nature de SCRUM, sa mise en place est beaucoup plus simple et peut être résumée par la figure suivante :

Le principe de base de SCRUM est le suivant :



Figure 7: la méthodologie scrum

- Dégager dans un premier lieu le maximum des fonctionnalités à réaliser pour former le BACKLOG du produit.
- En second lieu définir les priorités des fonctionnalités et choisir lesquelles seront réalisés dans chaque itération.
- Par la suite focaliser l'équipe de façon itérative sur l'ensemble de fonctionnalités à réaliser, dans des itérations appelées Sprints.
- Un Sprint aboutit toujours sur la livraison d'un produit partiel fonctionnel appelé incrément.

4.2.1.2 Planification d'un projet par SCRUM

Planification du Sprint : La planification s'appuie sur la planification de la « Release » réalisée en pente. La première réunion du sprint ne se limite pas à la planification, on y trouve également les activités suivantes :

- 1) Valider les « stories » du BACKLOG pris en compte dans le sprint et concevoir les solutions.
- 2) Identifier et estimer les tâches.

3) Prise des tâches par chacun des membres de l'équipe....

- Revue du Sprint : Permet de monter les résultats du développement effectués au cours du sprint, seule une version opérationnelle est montrée.
- Rétrospective : Faite en interne en équipe (avec la présence du SCRUM Master), l'objectif est de comprendre ce qui n'a pas bien fonctionné dans le sprint, les erreurs commises et de prendre des décisions pour procéder aux améliorations.
- SCRUM quotidien : Il s'agit d'une réunion de synchronisation de l'équipe de développement qui se fait debout en 15 minutes maximum au cours de laquelle chacun répond principalement à 3 questions :

- 1) Qu'est-ce que j'ai fait hier ?
- 2) Qu'est-ce que je ferais aujourd'hui ?
- 3) Quels sont les obstacles me retardant ?

4.2.1.3 Equipes et rôles

Une équipe [N3] Scrum est une petite équipe flexible qui se consacre à la livraison d'incréments de produits comités. Généralement petite (une dizaine de personnes), elle est tout de même suffisamment grande pour accomplir un volume de travail conséquent en un sprint. Une équipe Scrum doit rassembler trois rôles spécifiques : le Product Owner, le Scrum Master et l'équipe de développement.

- Le Product Owner (le propriétaire du produit) : c'est une personne qui porte la vision du produit à réaliser, généralement c'est un expert dans le domaine.
- Le SCRUM Master (le responsable de l'équipe SCRUM) : C'est la personne qui doit assurer le bon déroulement des différents sprints du release, et qui doit impérativement maîtriser SCRUM.
- Le SCRUM Team (l'équipe de SCRUM) : constitué des personnes qui seront chargées d'implémenter les différents besoins du client. Bien évidemment, cette équipe sera constituée des développeurs, des testeurs, etc.

Dans ce contexte de notre projet, monsieur Nouredine Loukil sera à la fois le propriétaire du produit en tant que chef du projet de l'entreprise et nous formons monsieur Ahmed Jemal, Sana et Eya les membres de l'équipe SCRUM.

5. Backlog Produit

Le backlog [N4] produit est la liste principale des tâches à réaliser. Il est géré par le Product Owner. C'est une liste dynamique de fonctionnalités, d'exigences, d'améliorations et de corrections qui fait office de point de départ pour le backlog de sprint. Pour résumer, c'est la « to-do-list » des équipes.

Priorité	Histoire d'utilisateur	Complexité
1	En tant qu'un visiteur ou un invité je peux envoyer une image ou un fichier.	Moyenne
2	En tant qu'invité je peux accéder directement au widget.	Moyenne
3	En tant qu'un visiteur je peux m'entrer mon nom et mon email pour accéder au widget.	Moyenne
4	En tant qu'un visiteur ou un invité je peux accéder à la page des FAQ.	Forte
5	En tant qu'un visiteur ou un invité je peux accéder à la page d'accueil.	Moyenne
6	En tant qu'un visiteur ou un invité je peux accéder à la page des messages.	Moyenne
7	En tant qu'un visiteur ou un invité je peux accéder à la page des nouvelles.	Moyenne

Tableau 3:backlog produit

Découpage en sprints :

Sprint	Histoire d'utilisateur	Date Début	Date fin
Sprint0 : Architecture Logicielle et Fonctionnelle	-Choix de l'architecture du projet. -Conception Globale -Fonctionnement du projet. - Installation de l'environnement. - Exécution du projet. -Présentation des outils et d'environnement logiciel.	01/02/2023	28/02/2023
Sprint 1 : Connexion et collecter les informations personnelles des utilisateurs	-En tant qu'un visiteur je peux accéder directement au widget (en envoyant mes informations personnelles). - En tant qu'un client je peux m'identifier en entrant mon nom et mon email (en envoyant mes informations personnelles).	01/03/2023	21/03/2023
Sprint 2 : Téléchargement d'une image ou d'un fichier	- En tant qu'un visiteur ou un client je peux télécharger une image ou un fichier.	22/03/2023	11/04/2023

Sprint 3 : Interface des questions fréquemment posées	- En tant qu'un visiteur ou un client je peux accéder à la page de FAQ's.	11/04/2023	01/05/2023
Sprint 4 : Interface des nouvelles	-En tant qu'un utilisateur je peux accéder à la page des nouvelles.	02/05/2023	15/05/2023
Sprint 5 : Création de l'interface d'accueil	-En tant qu'un utilisateur je peux accéder à la page d'accueil.	16/05/2023	29/05/2023

Tableau 4:Découpage en sprints

6. Conclusion

Ce chapitre présente une phase indispensable pour l'étude et l'analyse de notre application. Nous avons définies les principaux besoins fonctionnels et non fonctionnels. Nous avons aussi prouvé le choix de la méthodologie Scrum. Nous avons présenté le langage de modélisation utilisé. Et Nous avons fini par le Backlog du produit en spécifiant les différentes fonctionnalités qui le composent.

Dans le chapitre suivant, nous allons parler de l'architecture de notre web chat.

Chapitre II :

Sprint0 : Architecture Logicielle et Fonctionnelle

1. Introduction

Dans ce sprint, nous allons dévoiler la vue architecturale du système, présenter la conception générale du projet, définir le fonctionnement du projet, l'installation de l'environnement et la présentation des outils et d'environnement logiciel. Ceci nous permettra de bien assimiler les besoins déjà mentionnés dans le chapitre précédent.

2. Backlog Sprint 0

Le sprint est le cœur de SCRUM. Il s'agit d'un bloc de temps durant lequel un incrément du produit sera réalisé. Avant de se lancer dans un Sprint, l'équipe SCRUM doit obligatoirement définir le but de ce dernier qui doit être un tableau descriptif qui précise la charge du travail pour chaque tâche en nombre de jours, donc nous nous sommes intéressés à établir et définir notre sprint 0 est décrit dans le tableau suivant :

Taches	Acteurs	Priorité	Estimation
Développer l'architecture du système	Sana et Eya	élevé	12 jours
Définir le fonctionnement du projet	Sana et Eya	élevé	12 jours
Installation de l'environnement	Sana et Eya	élevé	12 jours
Présenter la conception générale du projet	Sana et Eya	élevé	12 jours
Présentation des outils et de l'environnement logiciel	Sana et Eya	Moyenne	4 jours
Tester le module	Sana et Eya	Moyenne	2 jours
Validation	Ahmed Jemal	Moyenne	1 Jour

Tableau 5: sprint 0

3. Vue architecturale du système

Nous avons choisi d'adopter une architecture multiniveau bien défini. Les interfaces du web chat, développées en React.js et l'application d'exacctly développé par Angular, permettent aux utilisateurs de visualiser les messages et d'interagir avec le chat. Le serveur principal, basé sur Spring Boot et connecté à une base de données PostgreSQL, gère le logique métier en gérant les communications, le traitement des messages, de question fréquemment posé (FAQ) et l'authentification des utilisateurs. Au niveau de persistance de données, un serveur secondaire développé en Node.js et connecté à une base de données MongoDB est spécifiquement dédié l'interface de nouvelles stockant les données correspondantes. Enfin

l'intégration avec Firebase permet le stockage des images et des fichiers envoyés par les utilisateurs.



Figure 8:architecture multiniveau

3.1Présentation d'Exacctlly

Exacctlly comprend plusieurs interfaces, mais pour notre projet, nous nous concentrons sur les interfaces essentielles comme l'interface des intégrations, interface de personnalisation de web chat et interface des messages échangés entre web chat et exacctlly.

3.1.1 Interfaces Graphiques

3.1.1.1 Interface des intégrations

Exacctlly offre une intégration fluide avec plusieurs applications, ce qui permet à ses clients d'améliorer leurs opérations de support clients en connectant leurs outils favoris, tels que Facebook, Instagram, Web chat et Presta shop.

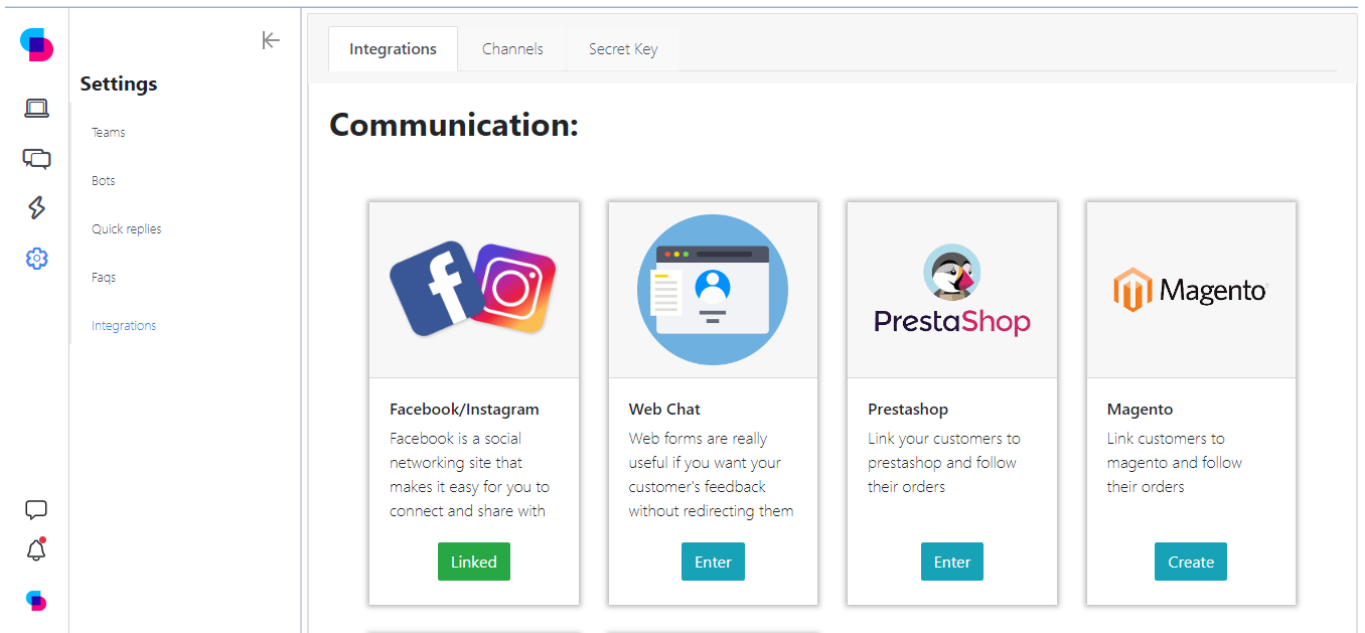


Figure 9: Interface des intégrations d'exacply

3.1.1.2 Interface de personnalisation du webchat

Exacply propose une interface pour la personnalisation du Web Chat, que ce soit au niveau du titre, du sous-titre ou des couleurs de l'en-tête, de l'envoi et de la réception de messages.

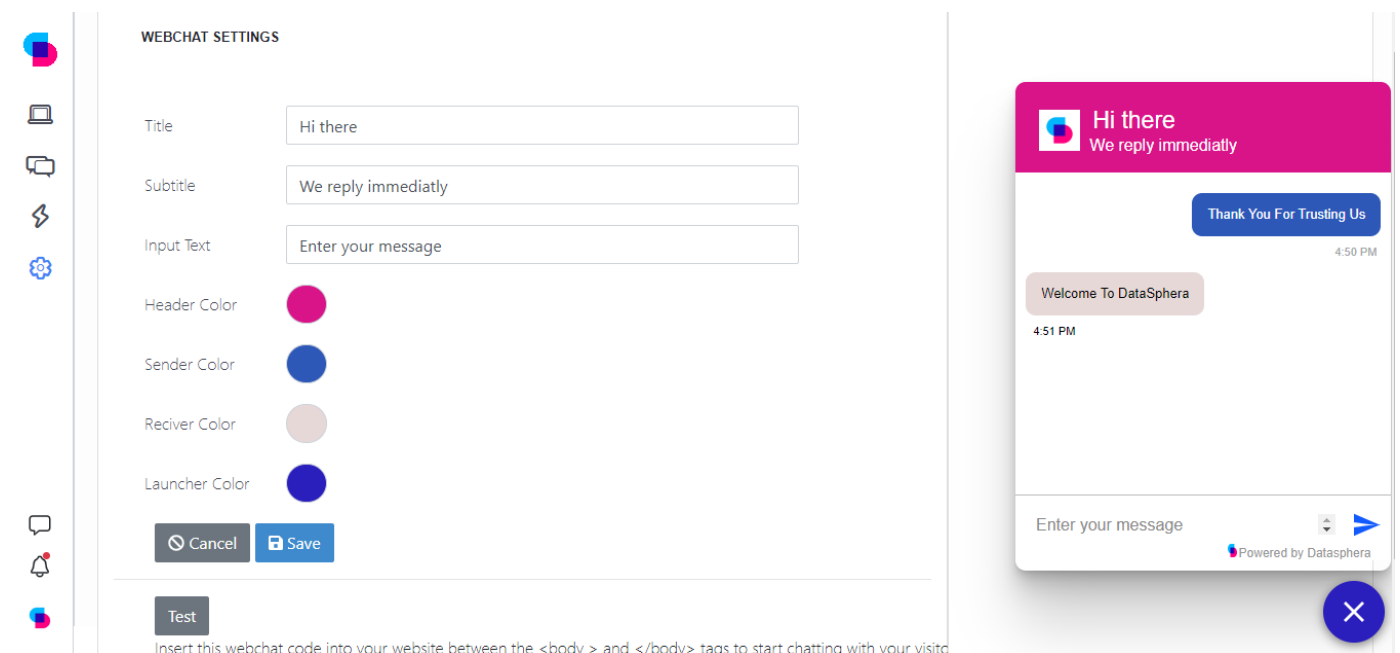


Figure 10: Interface de personnalisation du web chat

3.1.1.3 Interface des conversations échangées entre Exacctly et web chat

L'interface de conversations est une des interfaces essentielles d'Exacctly. Car elle est dédiée à la réception et au traitement des messages envoyés par les clients via le Web chat.

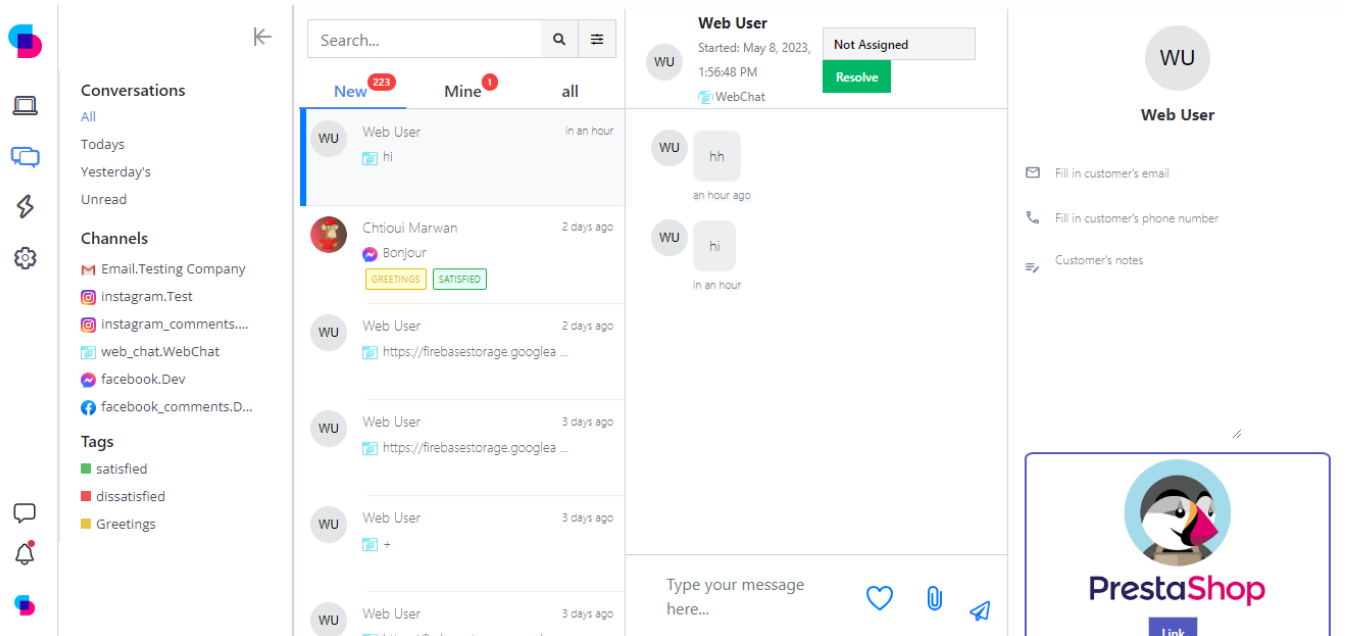


Figure 11:Interface de conversations

3.1.2 Web Chat Contrôleur

Le Web Chat Controller est un composant de l'application Spring qui agit en tant qu'intermédiaire entre l'application ReactJS et la base de données PostgreSQL. Son rôle est de gérer les requêtes provenant de l'application ReactJS et d'appeler les méthodes appropriées de la base de données pour les exécuter.

Le contrôleur est responsable de recevoir les requêtes HTTP provenant de l'application ReactJS, de les analyser et de les traiter en fonction de l'action demandée. Il utilise ensuite les méthodes appropriées de la base de données PostgreSQL pour effectuer les opérations correspondantes, telles que la récupération des messages via socket.io à partir d'Exacctly, puis l'envoi de ces messages à la base de données, l'insertion, la mise à jour ou la suppression de données.

3.2Présentation de l'interface du Widget

Data Sphera a apporté plusieurs modifications au widget de Rasa, notamment l'ajout d'un sélecteur d'emojis, d'un pied de page, d'un avatar dans l'entête et une option de personnalisation.

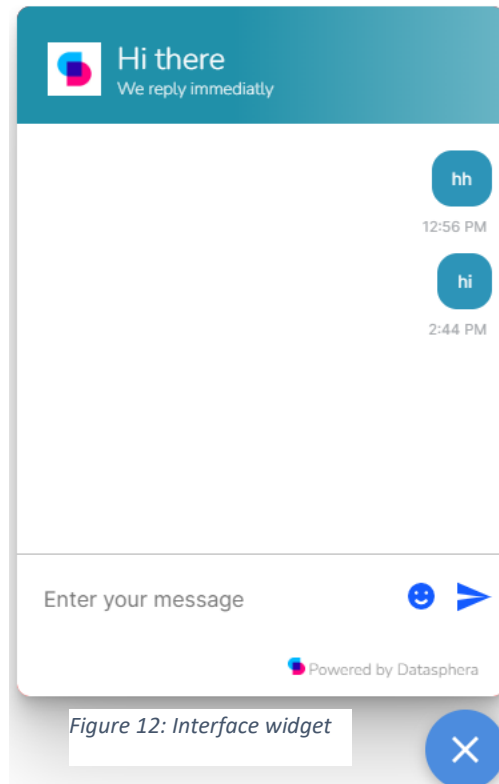


Figure 12: Interface widget

3.3Présentation Socket.IO

Socket.IO [N6] est une bibliothèque de communication en temps réel qui permet une communication bidirectionnelle et à faible latence entre un client et un serveur en utilisant le protocole Web Socket. Elle offre également un mode dégradé en HTTP long-polling pour garantir une disponibilité maximale de l'application.



Figure 13: Socket.io

Dans notre Widget on a 3 classes de Socket.IO que nous expliquons dans le tableau 6:

Socket	Est une classe qui agit comme une usine de création de socket. Elle permet de créer des instances de socket en utilisant différents protocoles, tels que socket.io et socket.js
---------------	---

Socket.IO	Est une classe qui utilise le protocole Web Socket pour établir une connexion en temps réel entre le client et le serveur via la bibliothèque socket.io-client. Le protocole Web Socket permet une communication bidirectionnelle entre le client et le serveur, ce qui permet d'envoyer et de recevoir des données en temps réel sans avoir à effectuer des requêtes HTTP répétitives.
Socket.js	Ce code implémente un client de communication en temps réel entre le client et le serveur à l'aide des protocoles Web Socket et HTTP basé sur AJAX. Il utilise la bibliothèque SockJS pour permettre la compatibilité avec les navigateurs qui ne prennent pas en charge nativement Web Socket. De plus, il utilise la bibliothèque STOMP pour la communication avec le serveur en utilisant le protocole STOMP. Cette classe gère la connexion, la réception et l'émission de messages, ainsi que la gestion de la déconnexion.

Tableau 6:explication de socket

4. Conception globale

4.1 Diagramme de cas d'utilisation global

Un cas d'utilisation (Use Case) représente un ensemble de séquences d'actions réalisées par le système et produisant un résultat observable et intéressant pour un acteur particulier. En effet, ce sont des représentations fonctionnelles du système, ils permettent de modéliser les attentes des utilisateurs afin de réaliser une bonne délimitation du système et également d'améliorer la compréhension de son fonctionnement.

La figure suivante présente le diagramme de cas d'utilisation global de notre projet :

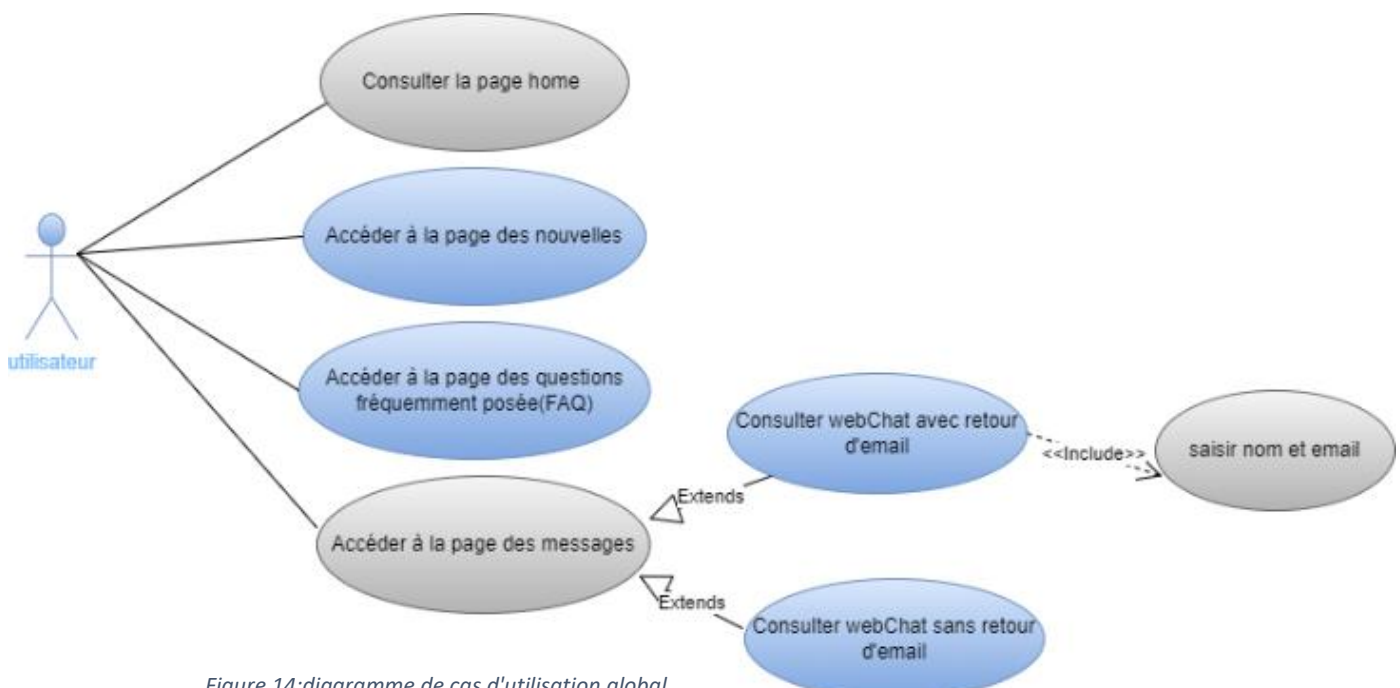


Figure 14:diagramme de cas d'utilisation global

4.2 Diagramme de classes

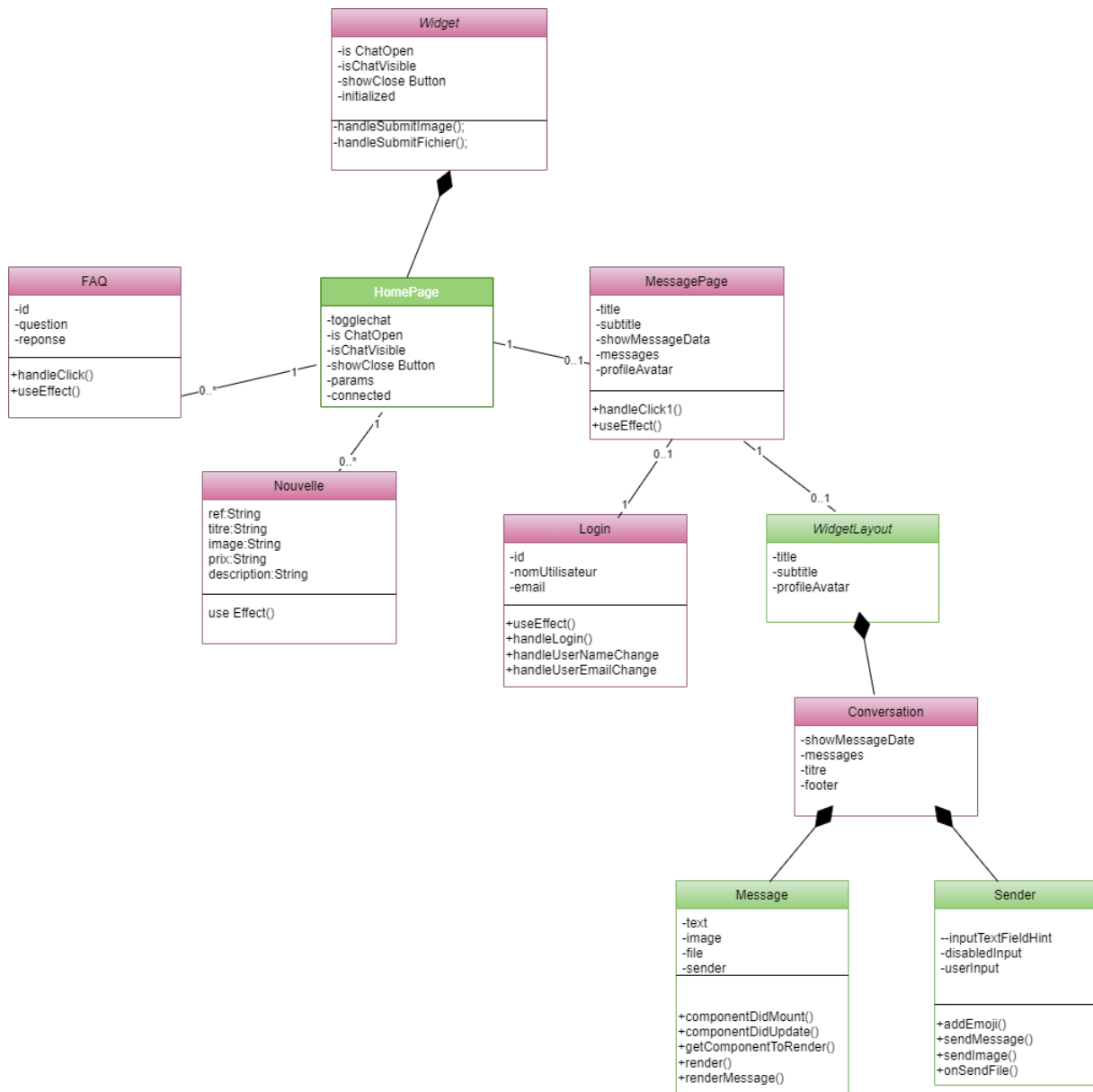


Figure 15:diagramme de classe global

Description de classes

Composant widget	C'est un composant responsable d'affichage notre application Il est composé d'un seul composant : HomePage
Composant HomePage	C'est un composant qui sert à afficher une interface d'accueil pour le web Chat.
Composant FAQ	C'est un composant qui implémente une foire des questions et réponses.
Composant Nouvelles	C'est un composant qui implémente les actualités dans le site web
Composant Message Page	C'est un composant responsable de l'affichage les deux boutons qui permet l'utilisateur d'accéder directement au chat s'authentifier
Composant Login	C'est un composant qui affiche un formulaire de connexion avec des champs de saisie pour que l'utilisateur entre son nom et son adresse e-mail.
Composant widget layout	C'est un composant responsable de gérer l'affichage et le comportement du widget de chat. Il est composé d'u seul composant : Conversation
Composant Conversation	C'est un composant qui crée une interface de conversation pour afficher différents éléments, tels que le titre, les messages et le pied de page.
Composant Message	C'est un composant qui contient une le message qu'il doit afficher dans le web chat.
Composant Sender	C'est un composant utilisé pour afficher le formuler d'envoi un message.

Tableau 7:Description des composants du diagramme de classe

4.3 Diagramme de séquence

Dans notre système, l'interface home page s'affiche. Lorsque l'utilisateur accède à la page des messages, le comportement diffère en fonction de son statut : les invités sont directement rédigés vers l'interface de chat, tandis que les visiteurs doivent se connecter en fournissant leur nom et leur email pour accéder à l'interface de chat. Les messages envoyés par les utilisateurs sont gérés par le contrôleur « webchatController » de l'application Exacctly.

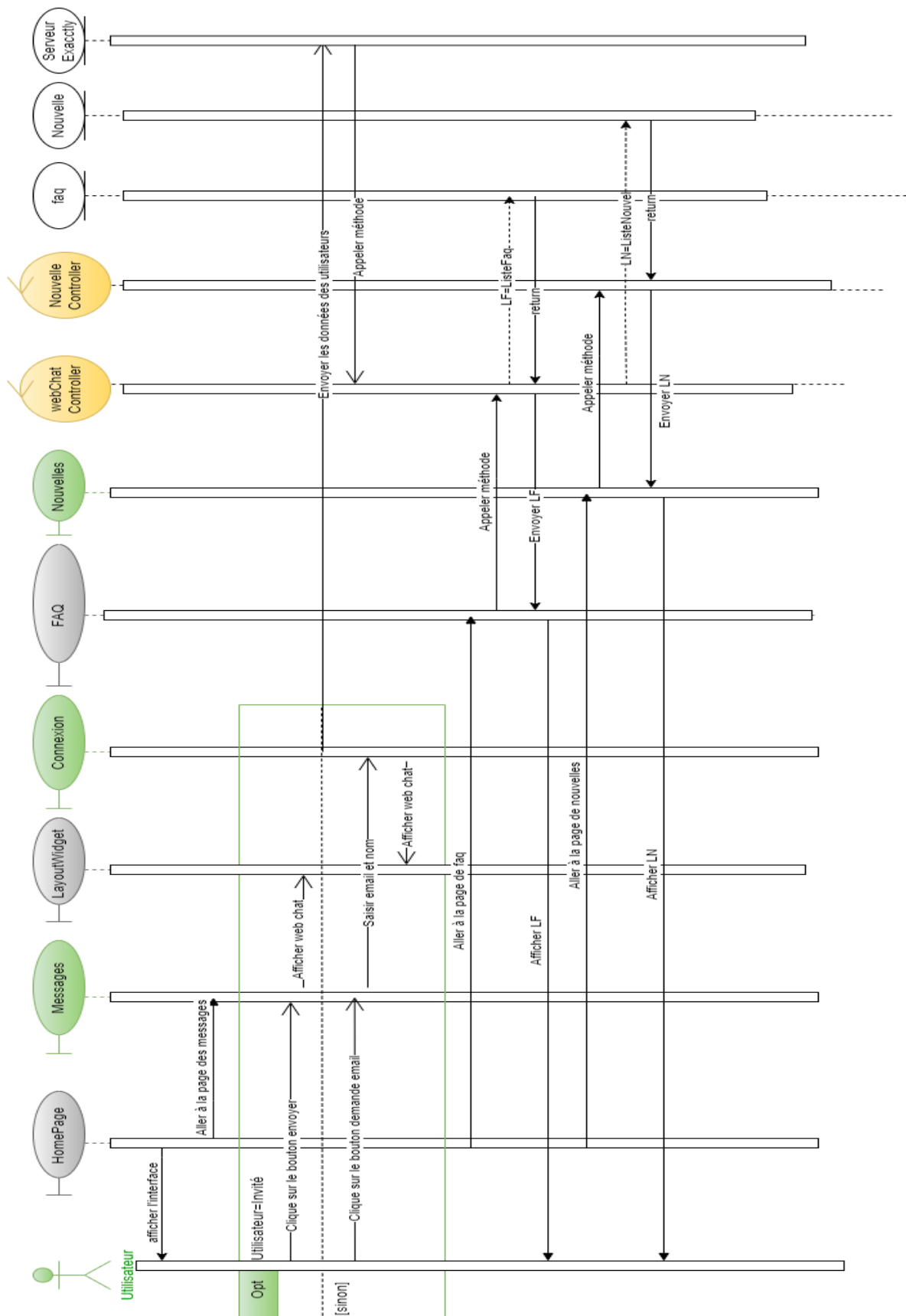


Figure 16:diagramme de séquence global

L'interface FAQ se connecte au webchatController pour récupérer les réponses aux questions fréquemment posées et les afficher. De même, la page de nouvelles se connecte au contrôleur « NouvelleController » pour récupérer les dernières nouvelles à afficher.

4.4 Diagramme de déploiement

Un diagramme de déploiement est une vue statique qui décrit l'architecture physique ainsi que les relations entre les composants logiciels et matériels d'une application et explique le déploiement de l'application en termes de réseau et de communication.

Dans le cadre de notre projet, nous avons mis en évidence les différentes composantes qui interagissent entre elles. Tout d'abord, nous avons le client web, qui représente l'interface utilisateur accessible via un navigateur web. Ce client web est connecté à une application frontend ReactJS via le protocole HTTP. À son tour, cette application ReactJS est connectée à une application Angular via une connexion socket.io, permettant ainsi un échange de données et d'événements en temps réel grâce au protocole Web Socket.

Ensuite, nous avons le serveur Spring qui joue le rôle de backend pour les deux applications Angular et ReactJS. Il héberge les services nécessaires pour le bon fonctionnement de ces applications. De plus, le serveur Spring utilise une base de données PostgreSQL pour le stockage et la récupération des données nécessaires aux applications.

Finalement, l'application ReactJS stocke les images et les fichiers dans Firebase, une plateforme de stockage de fichiers. En parallèle, pour la gestion de l'interface nouvelle, un serveur Node utilisant Node.js communique avec une base de données Mongo DB. Cela permet de stocker et récupérer les informations relatives à cette interface.

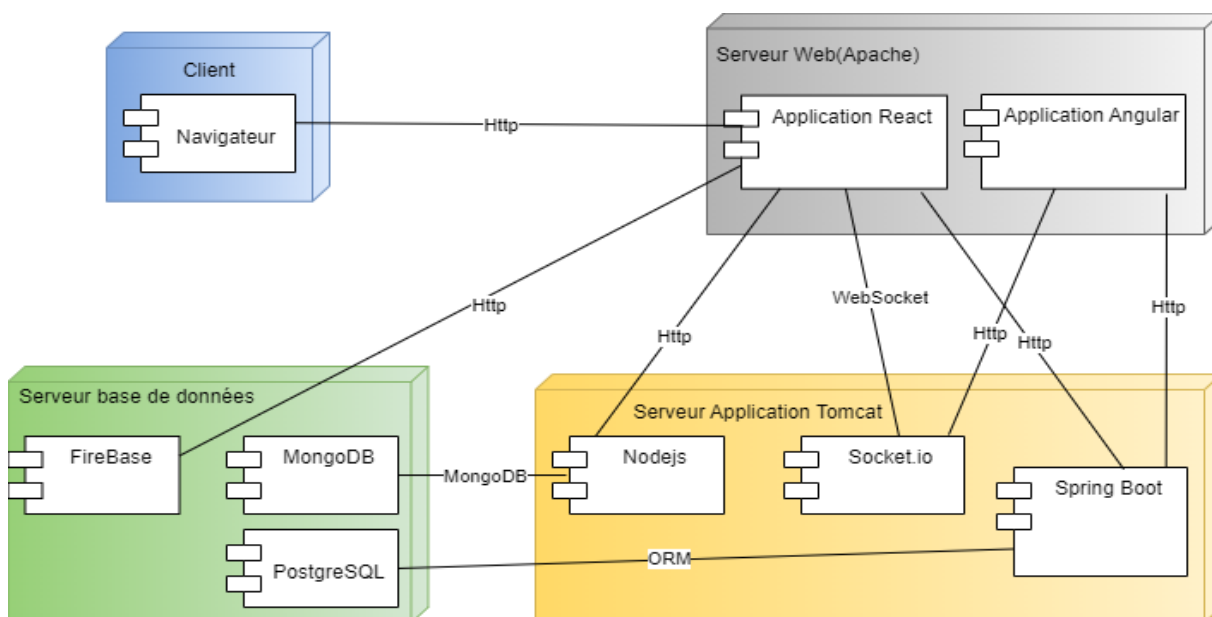
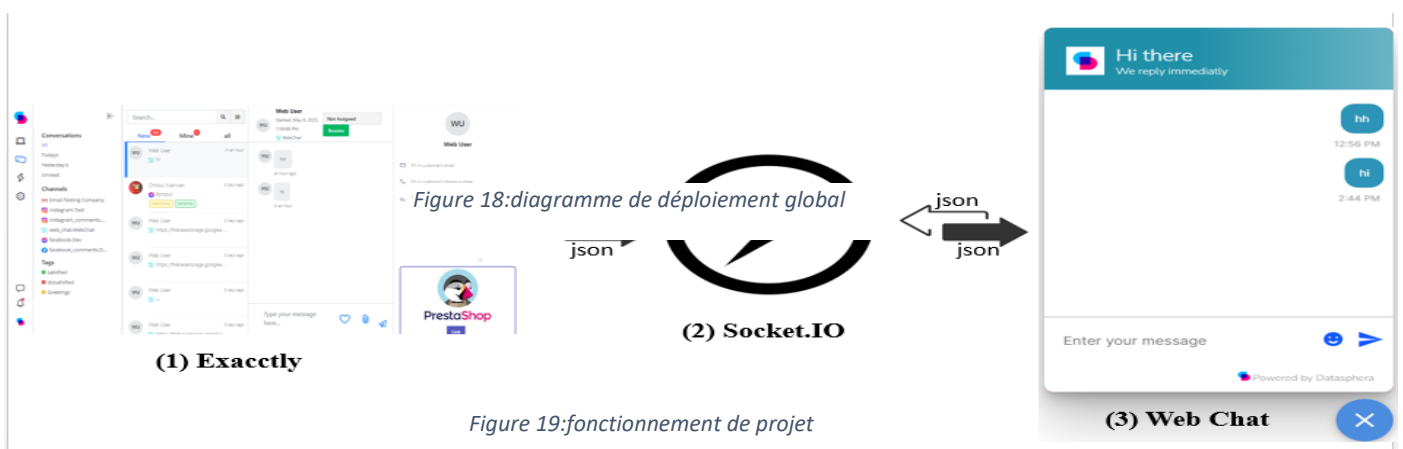


Figure 17:diagramme de déploiement

5. Fonctionnement

Notre web chat est relié à Exacctlly de Data Sphera via Socket.IO. Cette connexion permet de renforcer la relation entre l'entreprise et ses clients en résolvant leurs problèmes de manière plus rapide et efficace. En utilisant cette connexion, le web chat est capable d'interagir avec les bases de données de l'entreprise pour obtenir des informations précises sur les produits et les services, fournir des réponses instantanées aux demandes des clients, et offrir une expérience utilisateur plus personnalisée. Par conséquent, toutes les réponses et demandes sont envoyés sous format JSON entre les trois composants.



6. Installation de l'environnement du projet

Vu que notre travail consiste à améliorer une application existante en lui ajoutant de nouvelles fonctionnalités, nous étions dans l'obligation de récupérer le code existant et de l'exécuter localement sur nos machines avant de procéder à l'étape de rétro-ingénierie.

Du fait, nous avons suivi ce pipeline :



Figure 20: Etapes d'installation projet

6.1 Clonage du projet

Dans le cadre de notre projet, nous avons plusieurs étapes à suivre pour l'installation et le début du développement. Pour amorcer le processus, nous devons d'abord cloner le Web Chat à partir de Gitlab.

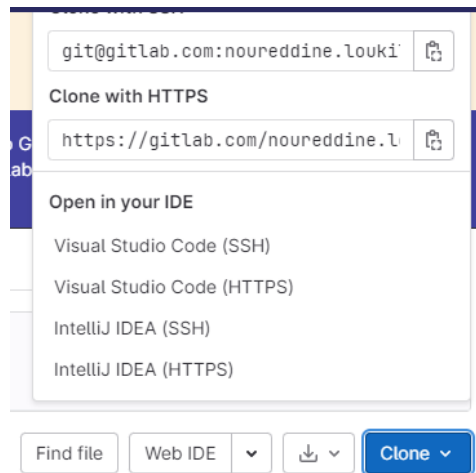


Figure 21: le lien de projet dans gitlab

Ensuite, nous pouvons ouvrir l'invite de commande et entrer simplement cette commande :

```
G:\>git clone https://gitlab.com/nouredidine.loukil/webchat-widget.git
Cloning into 'webchat-widget'...
remote: Enumerating objects: 6069, done.
remote: Counting objects: 100% (87/87), done.
remote: Compressing objects: 100% (56/56), done.
remote: Total 6069 (delta 46), reused 55 (delta 23), pack-reused 5982
Receiving objects: 100% (6069/6069), 6.34 MiB | 131.00 KiB/s, done.
Resolving deltas: 100% (3273/3273), done.
```

Figure 22: la commande de clonage dans visual studio code

6.2 Build de projet

Nous avons ouvert le projet Web Chat Widget dans Visual studio et avons modifié le lien de src dans le fichier test.html. Ensuite, nous avons utilisé la commande "npm run build" pour créer une version optimisée du code pour la production. Cette commande compile les fichiers JavaScript, CSS, HTML et images du répertoire de travail en un seul fichier JS, supprime les commentaires et les espaces blancs inutiles et réduit la taille des images. Après l'exécution de cette commande, un dossier lib est créé contenant le fichier index.js optimisé pour la production. Nous appelons ce fichier JS dans le fichier test.html et utilisons le pipeline de déploiement, comme indiqué dans le schéma présenté à la page ci-dessous :

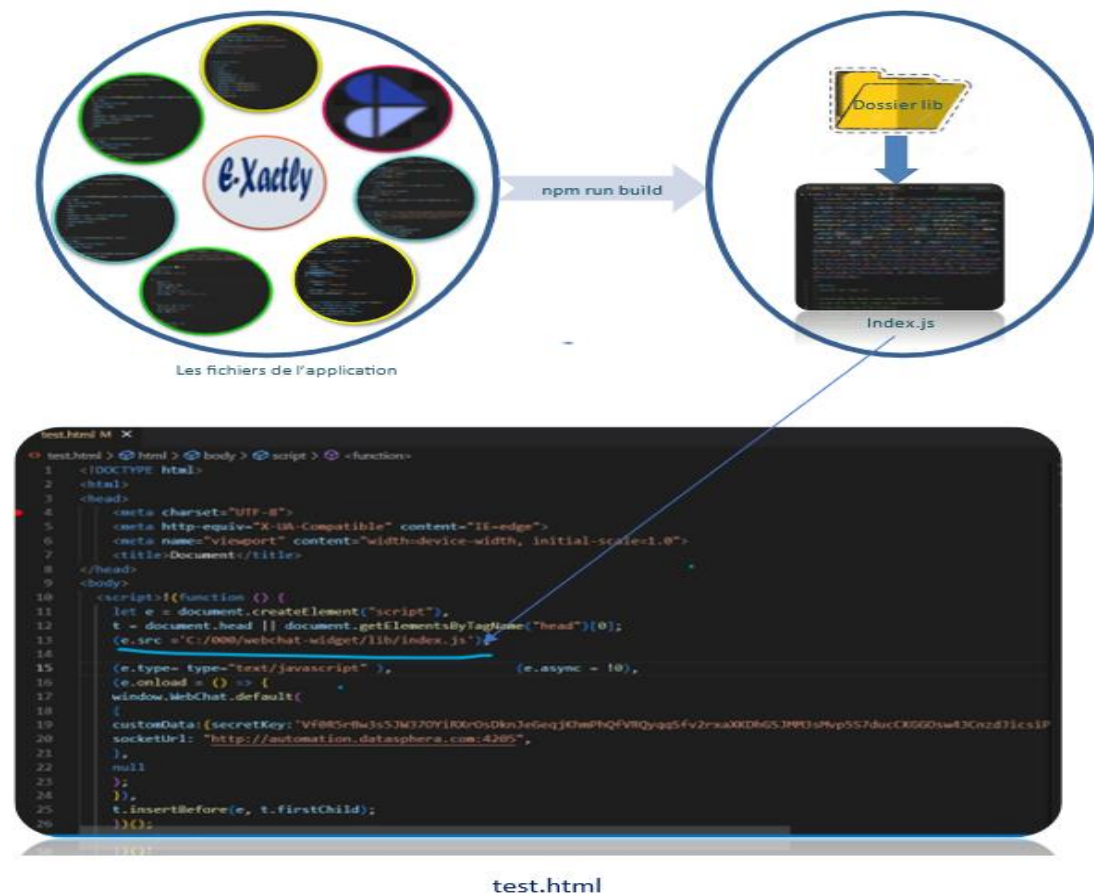


Figure 23:schéma explicatif du command de build

Nous avons eu un problème concernant le npm12 qui est déjà installé dans notre ordinateur. Nous avons installé npm6.

```

PS C:\pfe2023\webchat-widget> npm install -g npm@6.14.16
>>
  
```

Figure 24:la commande d'installation npm 6

Nous avons aussi un manque des dépendances donc nous utilisons cette commande pour l'installation des dépendances.

```

PS G:\webchat-widget> npm install
> rasa-webchat@1.0.4 prepare G:\webchat-widget
> npm run build
audited 2608 packages in 177.356s
59 packages are looking for funding
run `npm fund` for details
  
```

Figure 25: installation de dépendances

6.3 Résultat de l'exécution

Dans cette étape finale, nous avons exécuté le projet et l'interface du web chat s'est affiché avec succès.

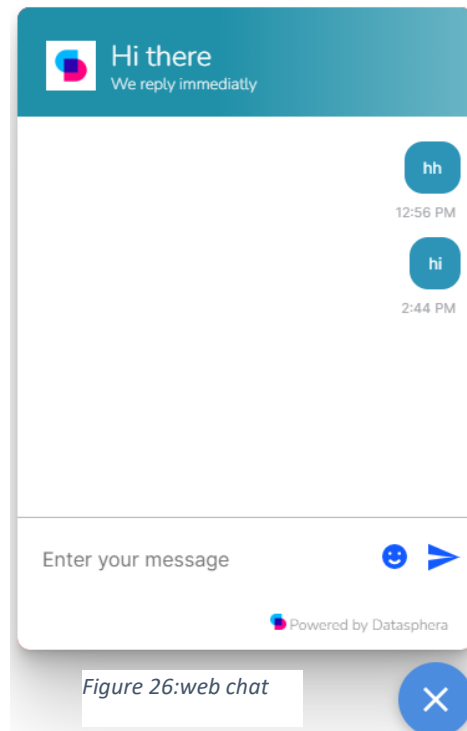


Figure 26:web chat

7. Outils et environnement logiciel

7.1 Environnement de développement intégré

Visual Studio Code est un éditeur de code extensible développé par Microsoft pour Windows, Linux et MacOS. Les fonctionnalités incluent la prise en charge de débogage, la mise en évidence de la syntaxe, la complétion intelligente du code, les snippets, la factorisation du code et Git intégré.



Figure 27:visual studio code

7.2 Système de Gestion de Base de données

7.2.1 Firebase

Firebase est une plateforme de développement d'applications mobiles et web de Google qui offre des fonctionnalités de backend évolutives et performantes. Elle permet la création des applications avec une base de données en temps réel. En tant que service cloud, Firebase élimine la nécessité pour les développeurs de gérer des serveurs et des infrastructures sous-jacentes.



Figure 28:firebase

7.2.2 Mongo DB

Mongo DB [N5] est une base de données orientée documents. En clair, vous bénéficiez de la scalabilité et de la flexibilité que vous voulez, avec les fonctions d'interrogation et d'indexation qu'il vous faut.



Figure 29:mongoDB

7.3 Framework

7.3.1 ReactJs

React est une bibliothèque JavaScript libre développée par Facebook depuis 2013. Le but principal de cette bibliothèque est de faciliter la création d'application web monopage, via la création de composants dépendant d'un état et générant une page HTML à chaque changement d'état.



Figure 30:reactJs

7.3.2 Node JS

NodeJs est un environnement d'exécution single-thread, open-source et multiplateforme permettant de créer des applications rapides et évolutives côté serveur et en réseau.



Figure 31:nodejs

7.3.3 Redux

C'est un paradigme de programmation qui a pour principe de structurer le code de façon à découper les problèmes en de multiples petites fonctions réutilisables.



Figure 32:Redux

7.4 Langages de programmation

7.4.1 Html

Le HyperText Markup Language, généralement abrégé HTML ou, dans sa dernière version, HTML5, est le langage de balisage conçu pour représenter les pages web.



Figure 33:html 5

7.4.2 CSS

La CSS (Cascading Style Sheets) est un langage de feuilles de style utilisé pour décrire la présentation et la mise en forme des pages web écrites en HTML ou XML.



Figure 34:css

7.4.3 Java Script

JavaScript est un langage de programmation interprété qui est principalement utilisé pour créer des pages web interactives et des applications web côté client.



Figure 35:java script

7.4.4 spring boot

Spring boot est outil qui accéléré et simplifie le développement d'applications web et de micros-services avec Spring Framework



Figure 36:spring boot

7.5 Outils de test

7.5.1 Postman

Postman est un logiciel gratuit qui permet d'appeler ou tester un API.



Figure 37:Postman

8. Conclusion

Lors de cette phase de développement, nous avons parlé de l'architecture, la conception globale, le fonctionnement, les installations requises pour notre projet et les outils et environnements logiciels.

Dans le prochain chapitre, nous aborderons le sprint1, qui se focalisera sur l'implémentation de la fonctionnalité de connexion utilisateur via une interface dédiée. Nous concentrerons également sur la collecte des informations personnelles des utilisateurs et la possibilité de télécharger des images ou des fichiers.

Chapitre III :

Sprint 1 et 2:

**Connexion et Collecter les données des
visiteurs**

**Téléchargement d'une image ou d'un
fichier**

1. Introduction

Ce chapitre présente deux fonctionnalités clés pour notre projet : une page des messages pour notre Web Chat et une page de connexion. La page des messages contient deux options. Quant à la page de connexion, elle permet aux clients d'enregistrer leur nom et adresse e-mail pour accéder au Web Chat.

Nous aborderons également le sujet du téléchargement d'images ou d'un fichier sur notre chat web en présentant les différents outils que nous utilisons.

Partie 1 : Sprint 1

2. Backlog du Sprint

Le sprint 1 présente les tâches suivantes :

Taches	Acteurs	Priorité	Estimation
Création des interfaces : messages et connexion	Sana et Eya	élevé	18 jours
Collecter les données des visiteurs et des invités.	Sana et Eya	élevé	20 jours
Tester le module	Sana et Eya	Moyenne	2 jours
Validation	Ahmed Jemal	Moyenne	1 jour

Tableau 8:backlog sprint 1

3. Analyse et spécification des besoins

3.1 Description

Les besoins à réaliser dans le sprint 1 étant spécifiés et afin de mieux les expliquer nous allons présenter le diagramme des cas d'utilisation, le diagramme de séquence et le diagramme de classes de l'authentification et des membres ainsi que leurs écrans IHM.

3.2 Diagramme des cas d'utilisations

Dans notre cas, la page d'accueil offre deux options pour accéder au widget, que ce soit en mode invité ou en mode connecté.

- **En mode Invité :**

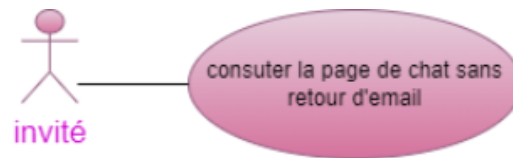


Figure 38:diagramme de cas d'utilisation en cas invité

- **En mode visiteur :**

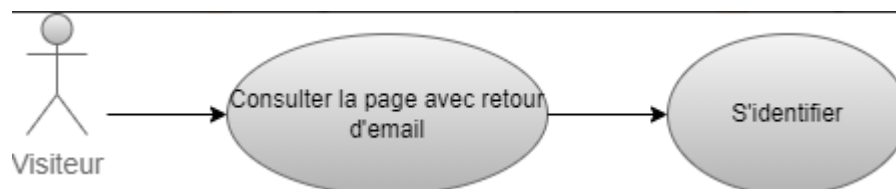


Figure 39:diagramme de cas d'utilisation en cas visiteur

3.2.1 Description textuelle des cas d'utilisations

3.2.1.1 Description Textuelle en mode invité

Cas d'utilisation	Invité
Acteur	Invité
Pré condition	Besoins d'accès à la page de widget.
Post condition	Succès d'accès.
Scénario nominal	1. L'invité accédé directement à la page de widget. 2. Les informations personnelles de l'invité sont envoyées à l'admin d'Exacctly (id session, IP, country, city, os)
Exception	E1 : Les informations ne sont pas envoyées, le widget ne s'affiche pas.

Tableau 9:description textuelle en mode invité

3.2.1.2 Description Textuelle en mode visiteur

Cas d'utilisation	Connexion
Acteur	Visiteur
Pré condition	Besoins d'accès à la page de connexion.
Post condition	Connexion validée et succès d'accès.
Scénario nominal	1. Le système affiche la page de connexion. 2. Le visiteur saisit son nom et son email. 3. Les informations personnelles du visiteur sont envoyées à l'admin d'Exacctly (id session, IP, country, city, os). 3. Le système vérifie que les champs de nom et d'email sont remplis, s'il est vides alors appel de l'exception « E1 ». 4. Le système donne l'accès au visiteur, la page widget s'affiche.
Exception	E1 : le formulaire n'est pas rempli. 1 : Affiche un message d'erreur

Tableau 10: description textuelle en mode visiteur

4. Maquettage

La maquette présente l'interface des messages du projet :



Figure 40:maquette interface messages

- Le bouton 1 intitulé « envoi-nous un message », ouvre directement le web chat.
- Le bouton 2 intitulé « Demande un Email », ouvre l'interface d'identification.

La maquette présente l'interface de connexion du projet :



Enter your name

Enter your email address

Start

Accueil Messages Aide Nouvelle

Figure 41:maquette d'interface de connexion

-A travers cette interface, l'utilisateur est possible de s'identifier et accéder à l'interface du web chat.

5. Conception

5.1 Diagramme de classes

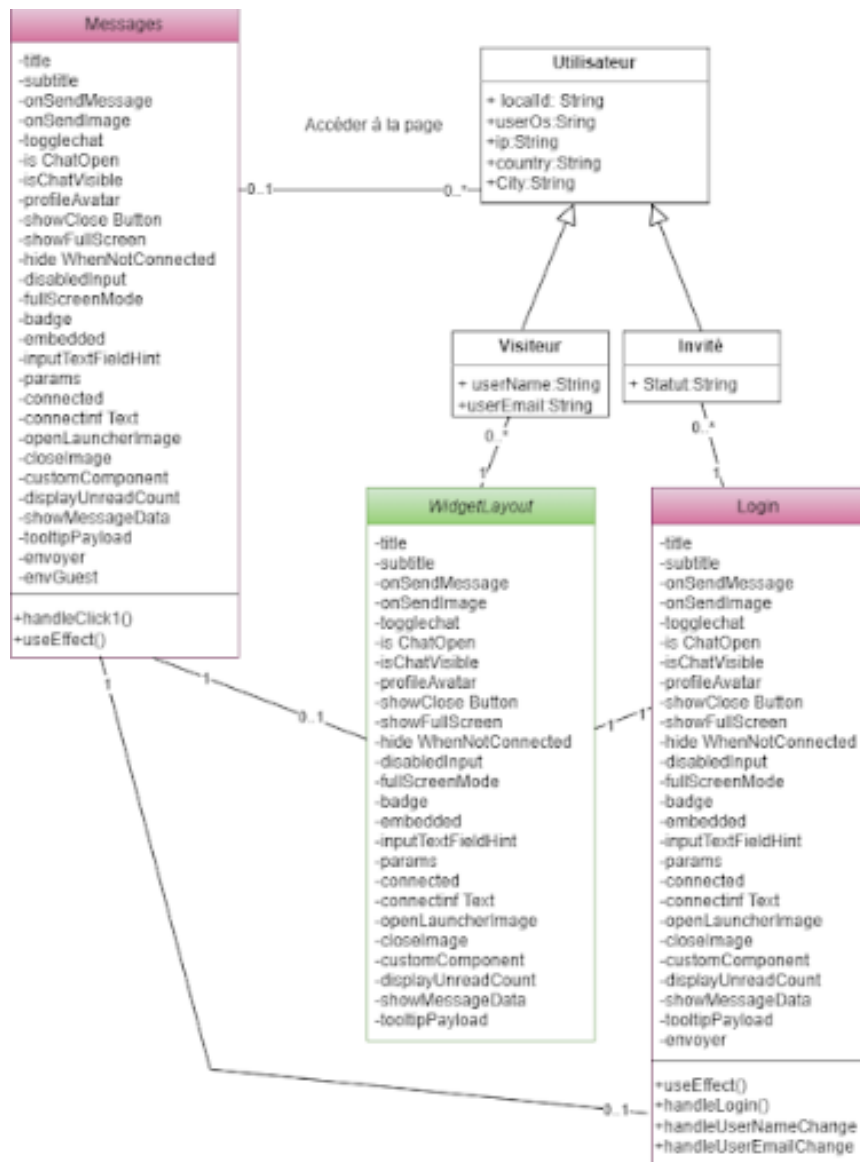


Figure 42: Diagramme de classes de la connexion

5.2 Diagramme de séquence

Dans notre diagramme de séquence, l'utilisateur peut ouvrir l'interface des messages. Le comportement varie en fonction de son statut. Les invités sont directement rédigés vers l'interface de chat, tandis que les visiteurs doivent se connecter en fournissant leur nom et leur email pour accéder à l'interface de chat. Les informations collectées par les utilisateurs sont gérés par le contrôleur « webchatController » du serveur Exacctly.

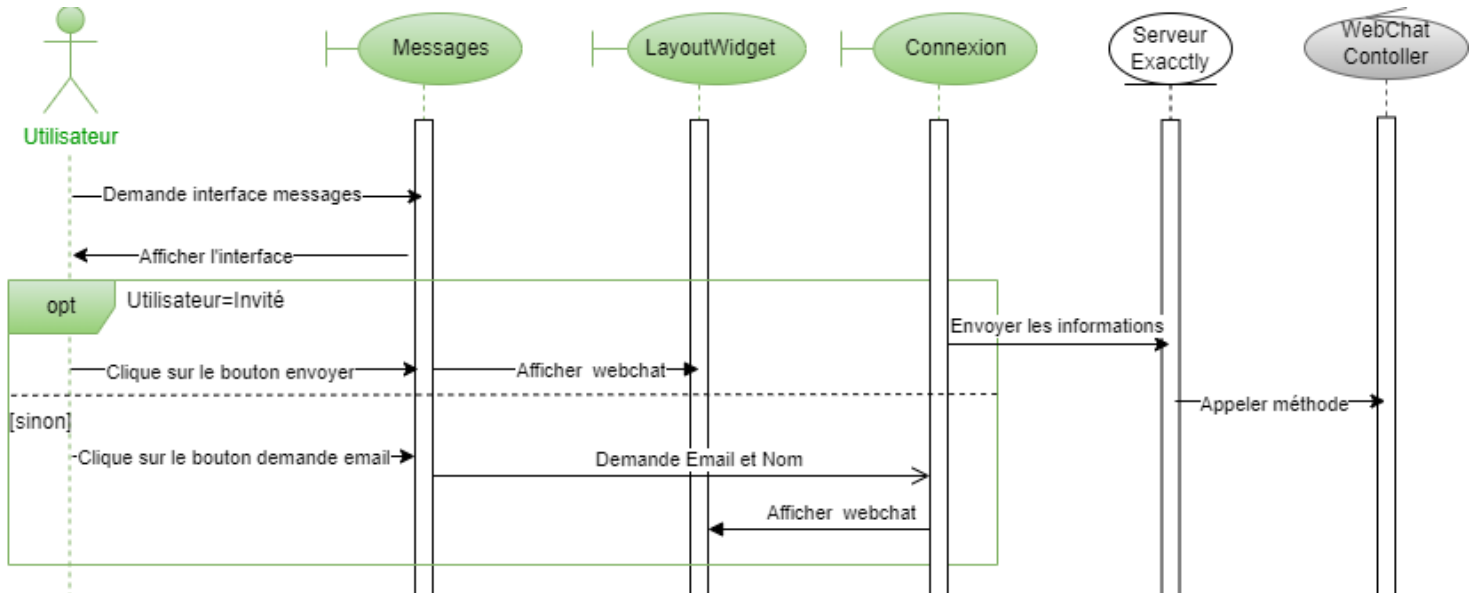


Figure 44:diagramme de séquence sprint 1

6. Réalisation

La phase de réalisation conclut le cycle de développement d'un sprint en présentant les résultats de l'étape de développement, ce qui permet de garantir une version de qualité. Pour illustrer cela, des captures d'écrans des fonctionnalités développées seront présentées.

➤ Interface des messages

Cette interface offre deux options : « invité » ou « visiteur ». Si vous êtes un invité, il suffit de cliquer sur le bouton « envoie-nous un message » pour accéder au web chat. En revanche, si vous êtes un visiteur, il faut cliquer sur le bouton « Demandé un e-mail » pour accéder à l'interface de connexion, où le web chat sera affiché.

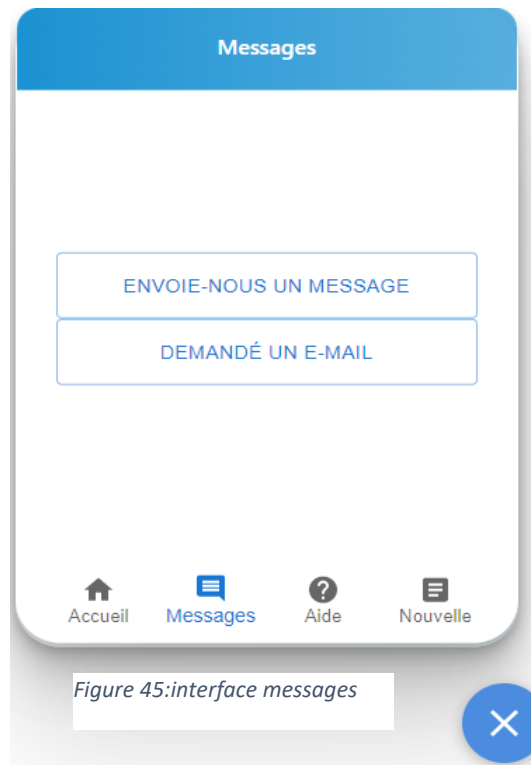


Figure 45: interface messages

Pour garantir une expérience utilisateur optimale sur le système Exacctly, deux méthodes importantes sont mise en places pour collecter et transmettre les informations des utilisateurs de manière sécurisée et confidentielle :

- **Si vous êtes un invité**

La fonctionnalité "envGuest" est mise en place sur le système Exacctly pour envoyer une instance de la classe "Guest" au serveur, si l'utilisateur est un invité, c'est-à-dire s'il n'est pas enregistré ou authentifié sur le système.

L'objectif de cette méthode est de collecter les informations personnelles des invités telles que son système d'exploitation, son adresse IP, son pays, sa ville et de les transmettre de manière sécurisée au serveur Exacctly pour une analyse plus poussée

```

async envGuset(userOS,ip,country,city,status){
  try {
    const data= new Guest(this.getSessionId(),userOS,ip,country,city,status);
    console.log("  localId  : "+data.localId+"   country  : "+data.country + "   ip  : "+data
let EXACCTLY_URL = 'https://418f-41-226-60-60.ngrok.io';
const serverResponse = await axios.post(`${EXACCTLY_URL}/webchat/send_Email`,data);
console.log("  data response  "+serverResponse.data);
} catch (error) {
  console.error(error);
}
}

```

Figure 46: la méthode envGuest

Pour comprendre le fonctionnement de la méthode "envGuest", nous avons réalisé une capture d'écran de la console côté serveur Exacctlly, qui montre les informations recueillies lorsqu'un invité utilise cette méthode pour envoyer ses données.

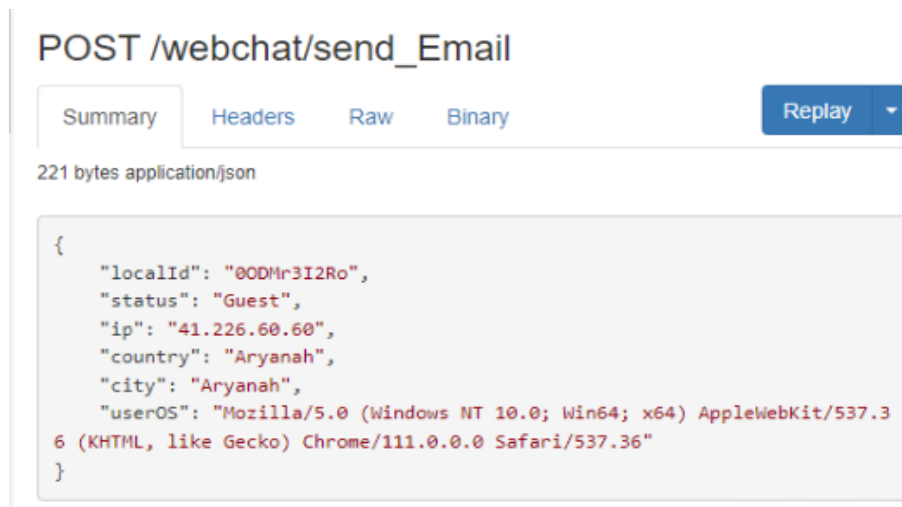


Figure 47: la réception de données de méthode "envGuest" coté exacctlly

- **Si vous êtes un visiteur**

La fonctionnalité "envoyer", a pour but de collecter et de transmettre les informations des utilisateurs connectés au serveur Exacctlly. Elle est essentielle pour recueillir des informations de base sur l'utilisateur, comme son nom, son adresse e-mail, son adresse IP, son pays, sa ville et son système d'exploitation.

Pour plus de détails sur le fonctionnement de la méthode "envoyer", veuillez-vous référer à la figure suivante :

```
async envoyer(userName, userEmail, ip, country, city, userOS){
  try {
    const data= new Envoy(this.getSessionId(),userName,userEmail,ip,country,city,userOS);
    console.log("  localId   : "+data.localId+"   country   : "+data.country + "   ip   : "+
    let EXACCTLTY_URL = 'https://418f-41-226-60-60.ngrok.io';
    const serverResponse = await axios.post(`${EXACCTLTY_URL}/webchat/send_Email`,data);
    console.log("  data response   "+serverResponse.data);
  } catch (error) {
    console.error(error);
  }}
}
```

Figure 48: méthode envoyer

➤ Interface de connexion

Dans l'exemple présenté ci-dessous, une connexion a été établie avec un utilisateur nommé "sana" ayant pour adresse e-mail "sana@gmail.com". Comme illustré dans la figure dans la page suivante :

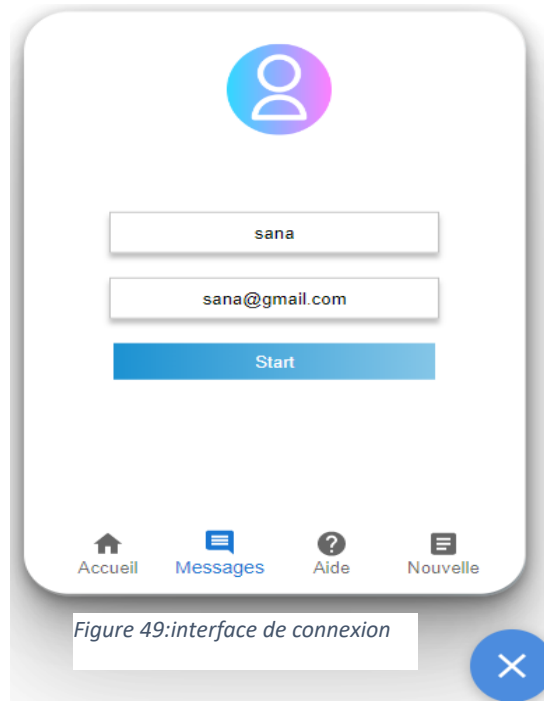


Figure 49:interface de connexion

Il est possible de constater que les informations de l'utilisateur "sana" sont bien reçues par le serveur Exacctly, comme le montre la figure :

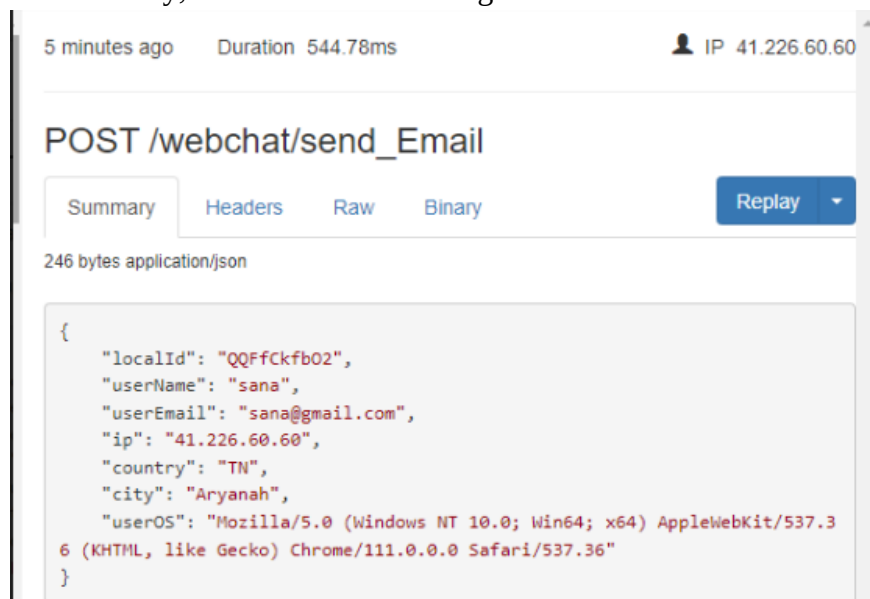


Figure 50:la réception de données de méthode "env" coté exactly

Partie 2 : Sprint 2

1. Backlog Sprint 2

Le sprint 2 présente les tâches suivantes :

Taches	Acteurs	Priorité	Estimations
Création d'un projet firebase et le configurer dans notre web chat	Eya et Sana	Moyenne	16 jours
Ajout d'un bouton 'téléchargement image' à notre interface web chat	Eya et Sana	Moyenne	12 jours
Programmation des méthodes nécessaires	Eya et Sana	Elevé	24 jours
Tester le module	Eya et Sana	Moyenne	2 jours
Validation	Ahmed Jemal	Moyenne	1 jour

Tableau 11:Backlog sprint 2

2. Analyse et spécification des besoins

2.1 Description

Les besoins à réaliser dans le sprint 2 étant spécifiés et afin de mieux les expliquer nous allons présenter le diagramme des cas d'utilisation, le diagramme de classe et le diagramme de séquence du téléchargement d'image ou du fichier ainsi que leurs écrans IHM.

2.2 Diagramme de cas d'utilisation

Dans notre interface web chat, on a l'accès de télécharger une image ou un fichier.

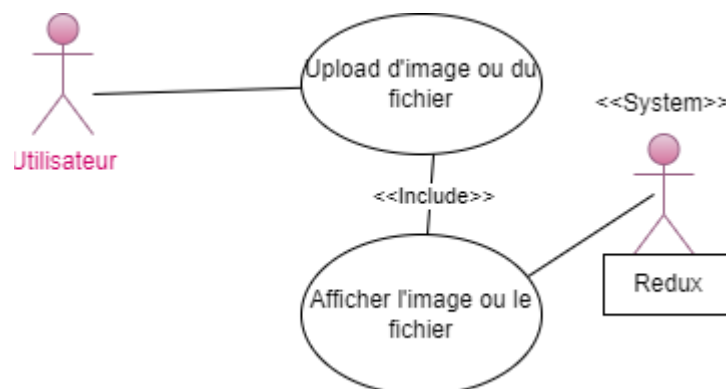


Figure 51:diagramme de cas d'utilisation sprint 2

2.2.1 Description textuelle du cas d'utilisation

Cas d'utilisation	Utilisateur
Acteur	Utilisateur
Pré condition	-L'utilisateur doit être authentifié et avoir les autorisations au Firebase. -Le web chat doit être connecté à Firebase.
Post condition	-L'image est téléchargée avec succès depuis Firebase. -L'image téléchargé est affiché à la fois sur web chat et Exacctly.
Scénario nominal	1. L'utilisateur sélectionne une image. 2. Configuration web chat avec Firebase. 3. Stocker l'image dans Firebase. 4. Firebase envoi le lien de l'image à web chat. 5. L'image s'affiche dans la zone d'envoi. 6. L'utilisateur envoie l'image. 7. l'image s'affiche sur la conversation du web chat et en Exacctly.
Exception	E1 : Expiration du projet Firebase. E2 : L'image sélectionner n'est pas stocker dans Firebase.

Tableau 12:decription de cas d'utilisation sprint 2

3. Schéma explicatif

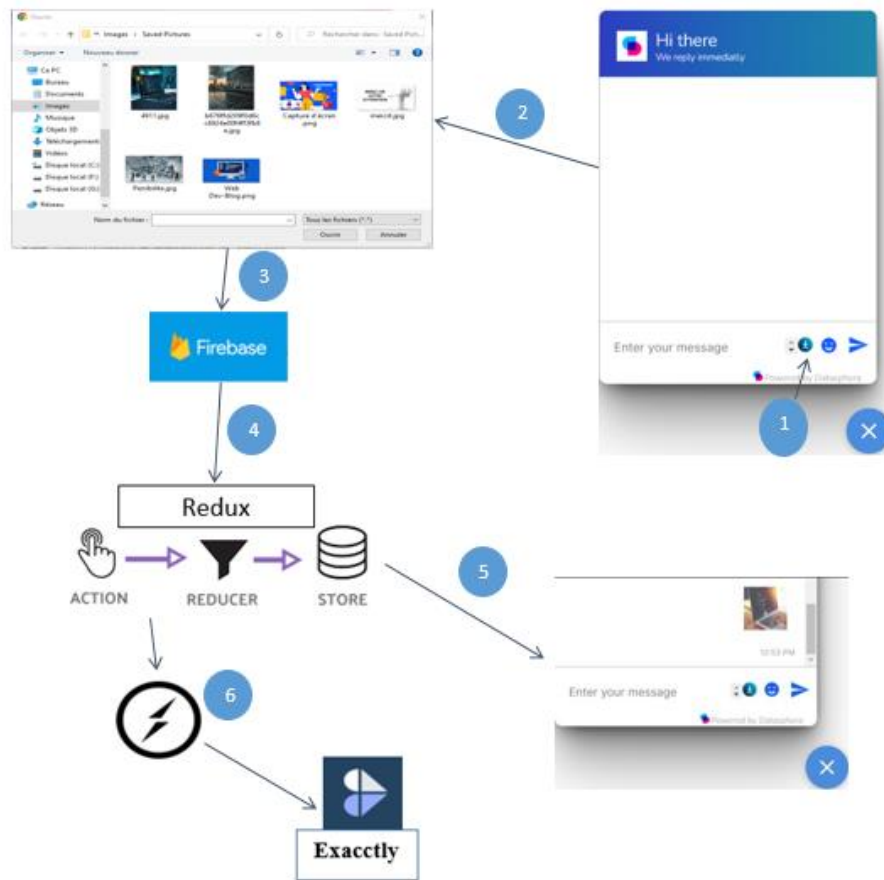


Figure 52:schéma explicatif

- 1) Cliquer sur l'icône de téléchargement.
- 2) Sélectionner une image sur le disque local et cliquer sur OK
- 3) L'image sélectionnée est stocké dans Firebase
- 4) Firebase envoie le lien de l'image vers l'action de Redux
- 5) Le fichier est stocké dans le store puis il est affiché dans le web chat
- 6) Le fichier est envoyé à exacctly via Socket.io

4. Maquettage

Dans la maquette de l'interface du web chat, on observe la présence du bouton 'télécharger une image':

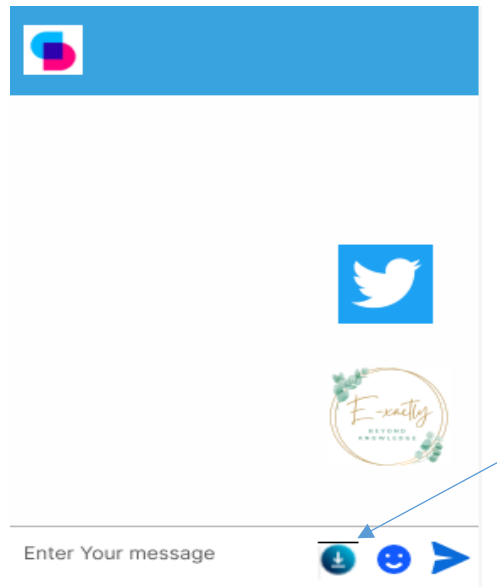


Figure 53:maquette de l'interface de bouton de téléchargement

4. Conception

4.1 Diagramme de classes

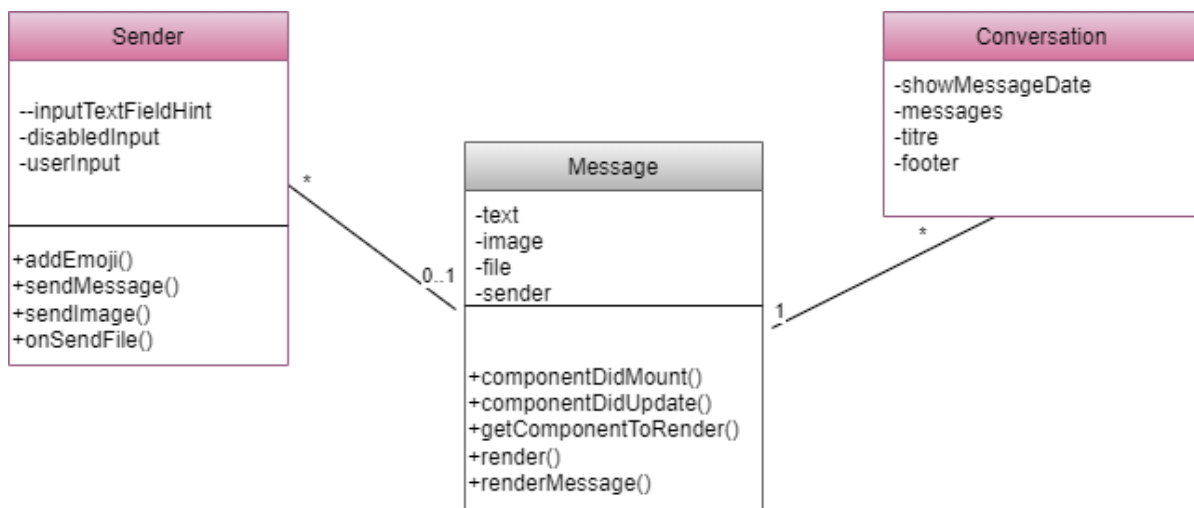


Figure 54:diagramme de classes sprint2

4.2 Diagramme de séquence

Dans ce diagramme de séquence, l'utilisateur doit sélectionner un fichier à partir du web chat. Ce fichier doit ensuite être stocké dans firebase, qui renvoie le lien de fichier au reducer. Le reducer envoie le lien de fichier au web chat pour qu'il puisse être affiché à l'utilisateur et envoie le lien à l'application exactement par le contrôleur web chat.

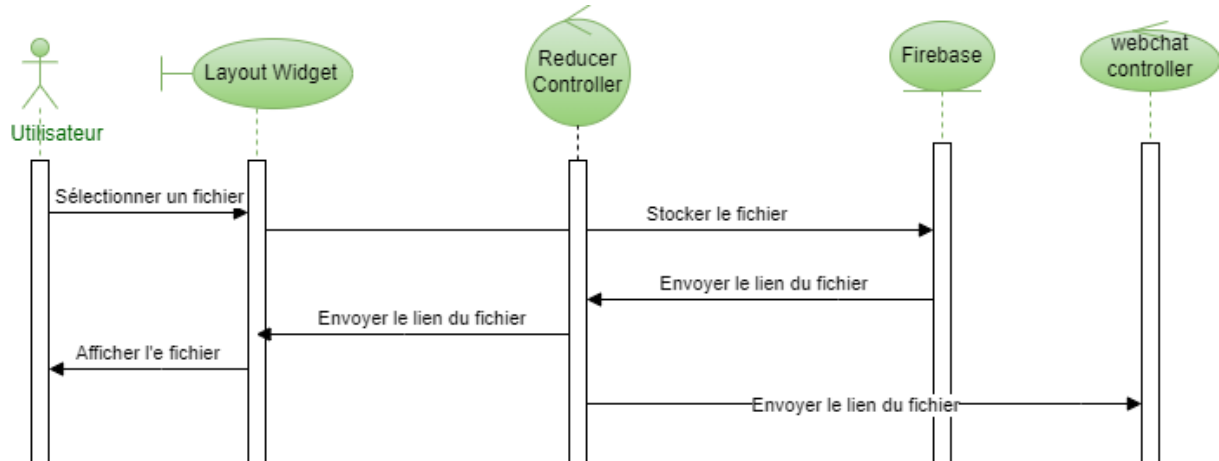


Figure 55:diagramme de séquence de sprint 2

5. Réalisation

Dans cette section, nous utilisons le service Firebase pour notre fonctionnalité de téléchargement d'image et de fichier. Donc, on a créé un projet FireBase :

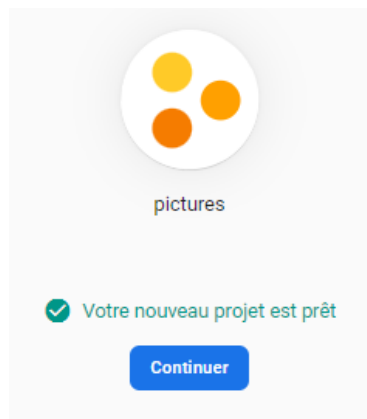


Figure 56:création de projet firebase

Nous initialisons le code de FirebaseConfig dans notre web chat pour pouvoir utiliser les fonctionnalités de stockage Firebase.

```
import { initializeApp } from 'firebase/app';
import { getStorage } from 'firebase/storage';

const firebaseConfig = {
  apiKey: "AIzaSyBV0RzhEk7_10B2iadq9iK8tot0tIbIV08",
  authDomain: "pictures-f2881.firebaseio.com",
  projectId: "pictures-f2881",
  storageBucket: "pictures-f2881.appspot.com",
  messagingSenderId: "728248821032",
  appId: "1:728248821032:web:246d43186be5d3c7955c1d",
  measurementId: "G-LVK5B9BM4X"
};

// Firebase storage reference
const app = initializeApp(firebaseConfig);
const storage = getStorage(app);

export default storage;
```

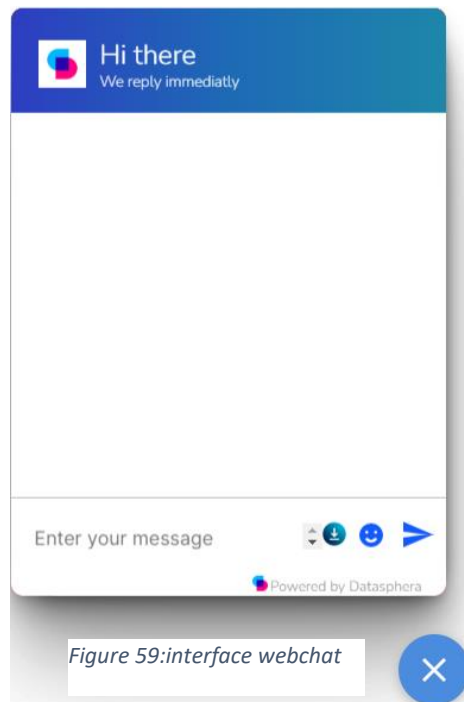
Figure 57:configuration de firebase

Nous ajoutons le code suivant dans l'index de Sender pour ajouter le bouton de télécharger d'image et du fichier :

```
<div>
  <div>
    <label htmlFor="image" className="file-input-label">
      <div className="placeholder-text">
        
      </div>
    </label>
    <input
      name="file"
      type="file"
      id="image"
      onChange={(e) => handleChangee(e)}
      style={{ display: 'none' }}
    />
  </div>
</div>
```

Figure 58:code de bouton télécharger

Alors l'interface du web chat affiche le bouton de téléchargement d'image et du fichier :



Explication de Redux

Dans notre phase de téléchargement d'un fichier, lorsque l'utilisateur effectue une action d'envoi un message avec une image ou un fichier, Redux est utilisé pour gérer cette action. Les données pertinentes, telles que le texte, l'image ou le fichier, sont encapsulées dans une action et transmises au "Reducer". Le reducer traite ensuite cette action et met à jour l'état global du web chat en ajoutant un nouveau message contenant les détails de l'image ou du fichier soumis. Ainsi, grâce à Redux, l'état de l'application est maintenu à jour et les nouvelles données sont correctement intégrées au flux de messages du chat web.

6. Conclusion

Dans ce Sprint, nous avons finalisé la conception et la mise en place des messages, de connexion et de téléchargement d'une image ou un fichier, qui a été validé par le SCRUM Master et le tester. Nous passons maintenant au prochain chapitre.

Chapitre IV

Sprint 3, 4 et 5 :

**Interface des questions fréquemment
posées**

Interface des nouvelles

Création d'interface accueil

1. Introduction

Dans ce chapitre, nous traitons les questions fréquemment posés dans notre web chat, ainsi que de la conception d'une interface de nouvelles et d'une autre interface d'accueil. Nous examinons en détail les différentes étapes impliquées dans la création de ces interfaces utilisateur.

Partie 1 : Sprint 3

2. Backlog du Sprint

Le sprint 3 présente les tâches suivantes :

Taches	Acteurs	Priorité	Estimations
Création interface de FAQ	Eya et Sana	Moyenne	8 jours
Programmation des méthodes nécessaires	Eya et Sana	Elevé	16 jours
Tester le module	Eya et Sana	Moyenne	2 jours
Validation	Ahmed Jemal	Moyenne	1 jour

Tableau 13:backlog sprint 3

3. Analyse et spécification des besoins

3.1 Description

Les besoins à réaliser dans le sprint 3 étant spécifiés et afin de mieux les expliquer nous allons présenter le diagramme des cas d'utilisation, le diagramme de séquence et le diagramme de classe du FAQ ainsi que leurs écrans IHM.

3.2 Diagramme de cas d'utilisation

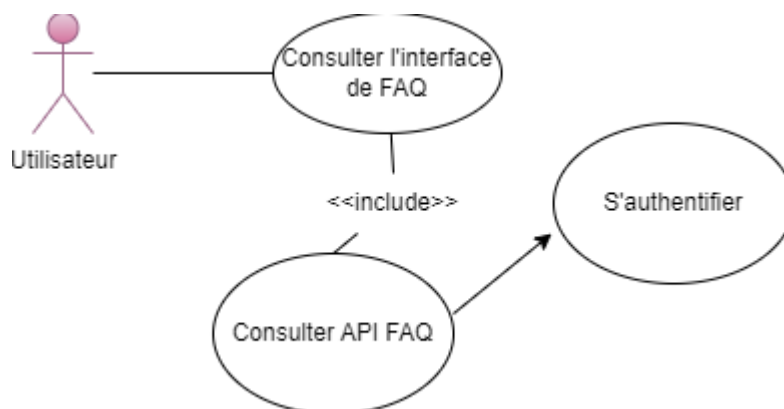


Figure 60:diagramme de cas d'utilisation sprint 3

3.2.1 Description textuelle du cas d'utilisation

Cas d'utilisation	Utilisateur
Acteur	Utilisateur
Pré condition	-Le web chat doit être connectée avec le token d'Exacctly.
Post condition	-Le FAQ du site Exacctly est récupéré avec succès et disponible pour être disponible dans le web chat.
Scénario nominal	1. L'utilisateur accède à l'interface FAQ. 2. Le système crée une connexion avec l'API d'Exacctly en utilisant le token fourni. 3. Le système envoie une requête à l'API pour récupérer la FAQ. 4. L'API renvoie le contenu de la FAQ au système. 5. Le système enregistre le contenu de la FAQ et l'affiché dans le web chat.
Exception	E1 : Le token est invalide. E2 : La connexion avec l'API d'Exacctly échoue.

Tableau 14: description textuelle du cas d'utilisation sprint 3

4. Maquettage

La maquette présente la liste de FAQ's :

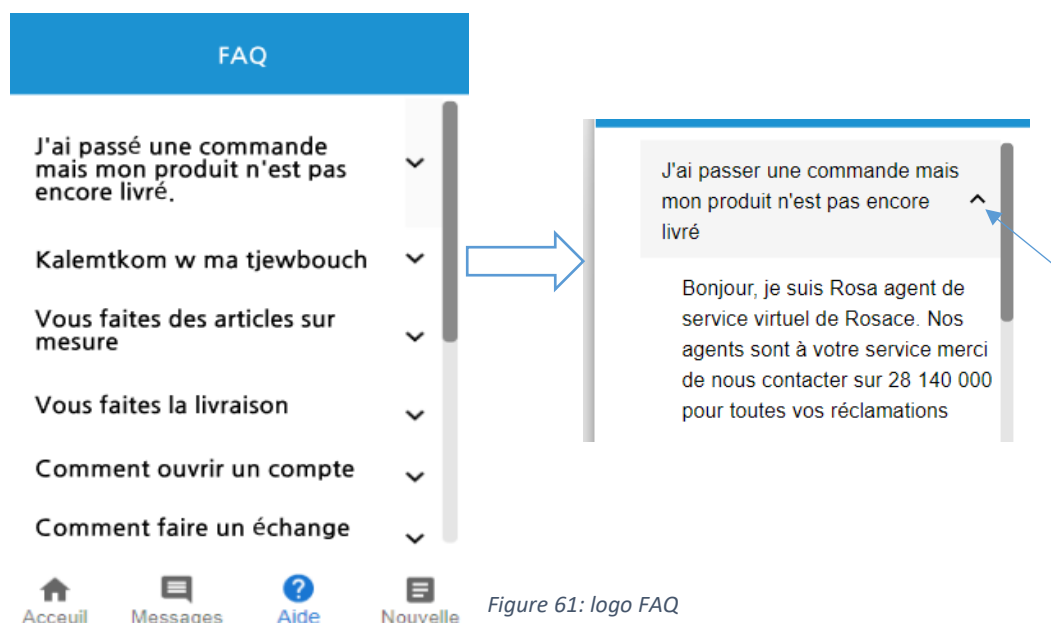


Figure 61: logo FAQ

Dans cette interface l'utilisateur consulte les questions proposées et afficher les réponses correspondantes en cliquant sur le bouton à droit de chaque question.

5. Conception

5.1 Diagramme de classes

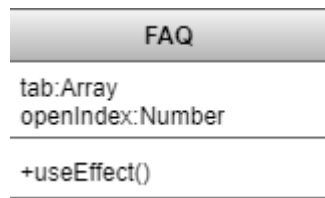


Figure 62: diagramme de classes sprint 3

5.2 Diagramme de séquence

Dans ce diagramme de séquence, Lorsque l'utilisateur accède à la page FAQ, l'interface se connecte au contrôleur d'Exacctly et appelle la méthode de récupération du tableau. Le contrôleur se connecte ensuite à la base de données et demande la liste des FAQ, qui est renvoyée par la base de données. Le contrôleur retourne ensuite cette liste à l'interface, qui l'affiche à l'utilisateur.

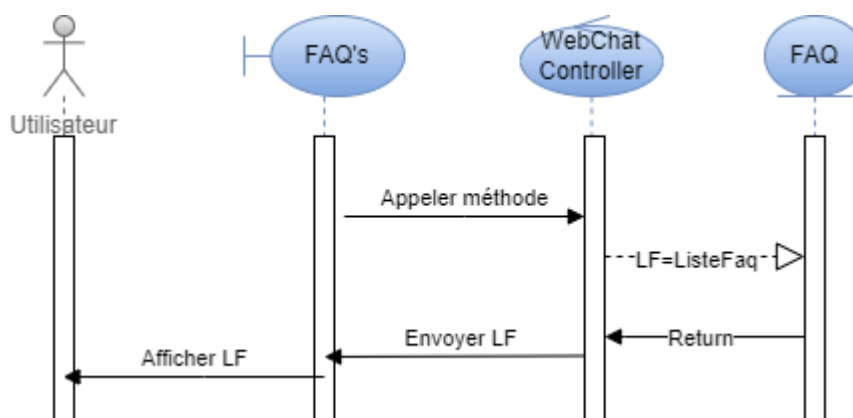


Figure 63:diag de séquence de faq

6. Réalisation

Au cours de cette phase, nous avons utilisé le token du site exacctly afin d'accéder à l'API de la FAQ.

Afin de vérifier le bon fonctionnement de l'api, nous avons récupéré token de l'application exacctly comme précise dans la figure 62 :

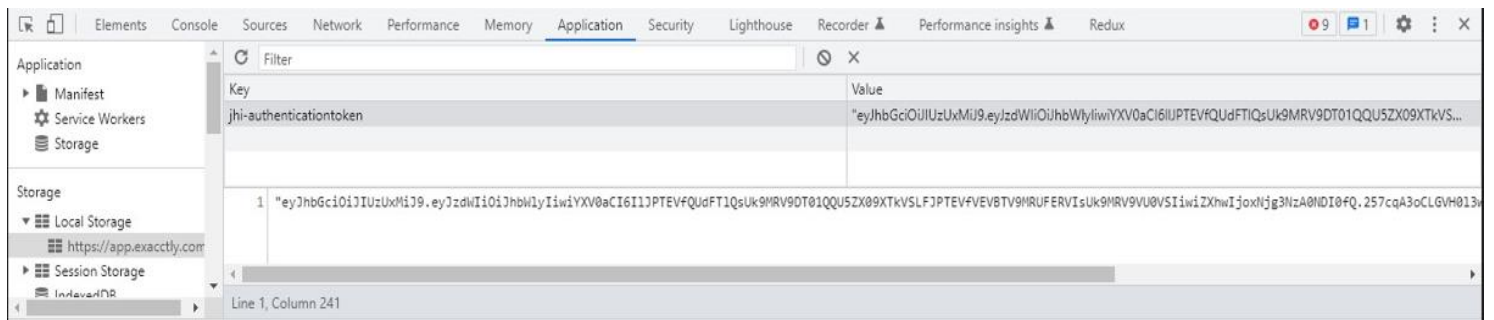


Figure 64: récupération de token de site exactly

Ainsi, nous effectuons des tests sur l'API à l'aide de Postman

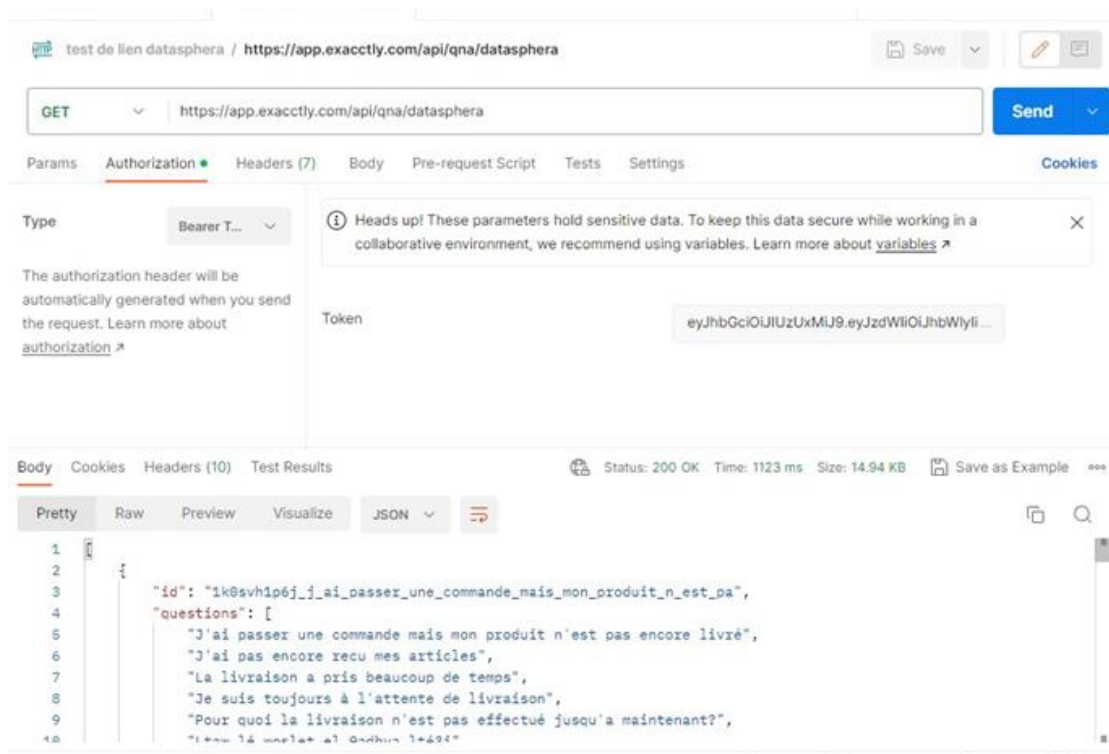


Figure 65: test de l'api avec postman

Enfin, nous avons ajouté un code qui implémente une fonction appelée 'FetchData'. Cette fonction envoie une requête POST pour s'authentifier dans l'application Exactly et récupérer

Le token d'accès. Ensuite, elle envoie une requête GET à l'url '**https://app.exactly.com/api/qna/datasphera**' avec un en-tête d'autorisation contenant le jeton d'accès. Donc, Cela nous permet d'afficher les questions-réponses du FAQ. Les données sont récupérées à partir d'une API en utilisant la bibliothèque axios. De plus, les questions peuvent être ouvertes ou fermées en cliquant dessus.


```

useEffect(() => {
  const fetchData = async () => {
    try {
      const username = ' ';
      const password = ' ';
      const data={
        username:username,
        password:password
      }
      const faq = await axios.post('https://app.exacctly.com/api/authenticate',data);
      const token=faq.data.id_token;
      console.log('Token:', token);
      const headers={
        'Authorization':'Bearer ' + faq.data.id_token
      }
      const res= await axios.get('https://app.exacctly.com/api/qna/datasphera',{ headers }
    )
      setTab(res.data)
      console.log(res.data);
    } catch (error) {
      console.error(error);
    }
  };
  fetchData();
}, []);

```

Figure 66:code faq

En fin de compte, l'interface de la FAQ se présente dans la figure:

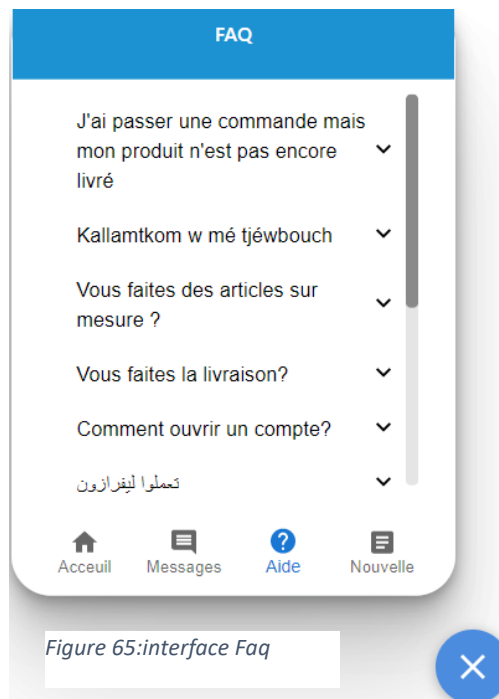


Figure 65:interface Faq

Partie 2 : Sprint 4

1. Backlog du Sprint

Le sprint 4 présente les tâches suivantes :

Taches	Acteurs	Priorité	Estimation
Création du backend de nouvelles	Sana et Eya	Moyenne	8 jours
Création de l'interface	Sana et Eya	Moyenne,	4 jours
Programmation de méthodes nécessaire	Sana et eya	Moyenne	12 jours
Tester le module	Sana et Eya	Moyenne	2 jours
Validation	Ahmed Jemal	Moyenne	1 jour

Tableau 15:backlog sprint4

2. Diagramme de cas d'utilisation

2.1 Diagramme de cas d'utilisation

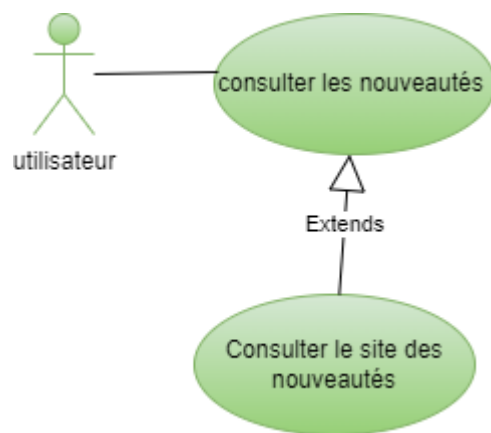


Figure 66:diagramme de cas d'utilisation sprint

2.2 Description textuelle du cas d'utilisation

Cas d'utilisation	Utilisateur
Acteur	Utilisateur
Pré condition	L'application web doit être connectée avec le serveur
Post condition	La liste des nouvelles est affichée avec succès.
Scénario nominal	1. L'utilisateur accède à l'interface des nouvelles. 2. Connexion de l'application web avec le serveur 3. affichage de la liste des nouvelles 4. l'utilisateur clique sur une nouvelle 5. la nouvelle sélectionnée s'ouvre dans le site.
Exception	E1 : L'application web n'est pas connectée avec le serveur E2 : Problème dans le serveur

Tableau 16: description de cas d'utilisation sprint 4

3. Maquettage :

La maquette présente l'interface des nouvelles :



Figure 67: maquette de l'interface nouvelles

4. Conception

4.1 Diagramme de classes:

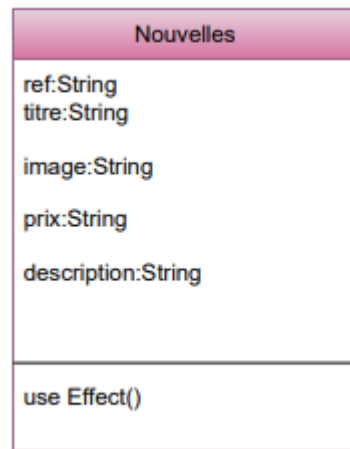


Figure 68:diagramme de classes sprint 4

4.2 Diagramme de séquence :

Dans ce diagramme de séquence nous faisons appel à une liste de nouvelles depuis un Controller. Ainsi, ce dernier retourne la liste à l'interface pour l'afficher à l'utilisateur.

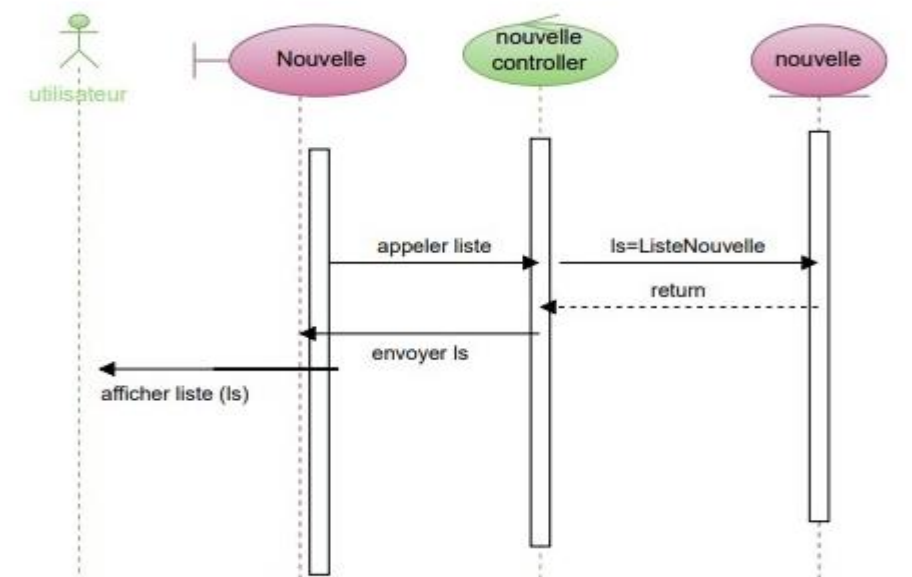


Figure 69:diagramme de séquence sprint 4

5. Réalisation :

Dans cette section, nous mettons en place un backend en utilisant Node.js et Mongo DB. Nous suivons l'architecture MVC (Modèle-vue-Contrôleur) pour organiser notre code.

Pour commencer, nous créons un module en utilisant le package Mongoose. Ce module nous permet d'interagir avec la base de données Mongo DB. Ensuite, dans le contrôleur, nous

implémentons la méthode permettant de récupérer toutes les données de table « nouvelles ». Dans le routeur, nous définissons une route correspondant à cette méthode de récupération. Enfin, dans le fichier app.js nous publions cette API du serveur en utilisant le Framework Express.

Nous disposons d'une capture d'écran représentant la structure de la classe « nouvelles » :

```
1  import mongoose from 'mongoose';
2  var nouvelleChema=mongoose.Schema({
3      ref:String,
4      image:String,
5      prix:String,
6      titre:String,
7      description:String,
8      genre:String,
9      lien:String
10 })
11 const Nouvelle=mongoose.model("Nouvelle",nouvelleChema);
12 export default Nouvelle;
```

Figure 68:structure de la classe « nouvelles »

Alors, nous avons une représentation visuelle de la table dans Mongo DB :

The screenshot shows the MongoDB Compass interface for the 'nouvelles' collection. The top bar indicates 10 documents and 1 index. The 'Documents' tab is selected, showing a list of documents. The first document has a ref field pointing to '5B00030' and a lien field pointing to 'https://rosace.com.tn/uploads/articles/Sabots%20en%20Halfa%20%C3%A0%20...'. The second document has a ref field pointing to 'F28' and a lien field pointing to 'https://rosace.com.tn/uploads/articles/Sabots%20%C3%A0%20large%20bride...'. The interface also includes a search bar, a filter button, and a 'More Options' button.

Figure 69:la table "nouvelles " dans Mongo DB

Dans la partie frontend de notre application, nous utilisons le composant « Nouvelle » pour effectuer une requête à l'API du serveur et récupérer les données associées. Les données sont

ensuite stockées dans un tableau pour un traitement ultérieur. Pour avoir un aperçu visuel de cet affichage, vous pouvez consulter l'image dans la page suivante :

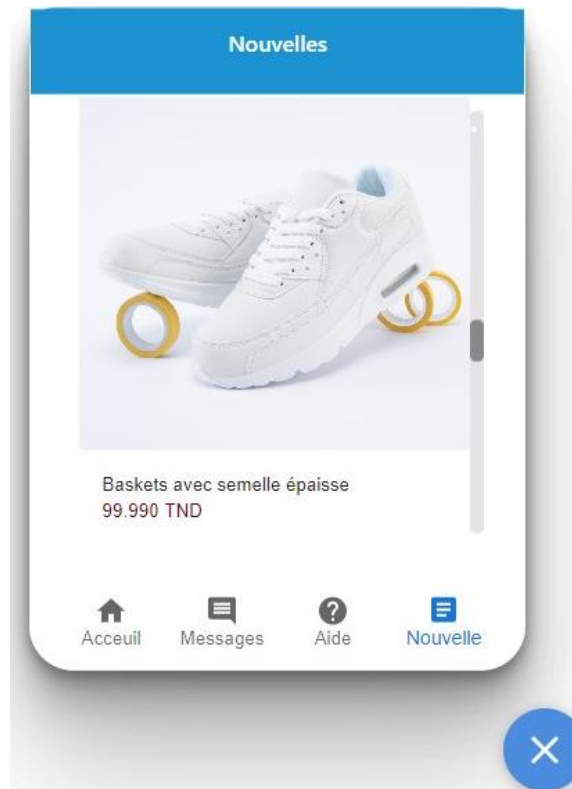


Figure 70:interface nouvelles

Lorsque l'utilisateur clique sur une nouvelle, celle-ci s'ouvre sur le site comme vous pouvez le voir dans l'image suivante :



Figure 71: clique sur un nouvelle

Partie 3 : Sprint 5

1. Backlog du Sprint

Le sprint 5 présente les tâches suivantes :

Taches	Acteurs	Priorité	Estimation
Création de l'interface d'accueil	Sana et Eya	Moyenne	12 jours
Programmation de méthodes nécessaire	Sana et Eya	Moyenne	12 jours
Tester le module	Sana et Eya	Moyenne	2 jours
Validation	Ahmed Jemal	Moyenne	1 jour

Figure15:backlog sprint 5

2. Diagramme de cas d'utilisation

2.1 Diagramme de cas d'utilisation

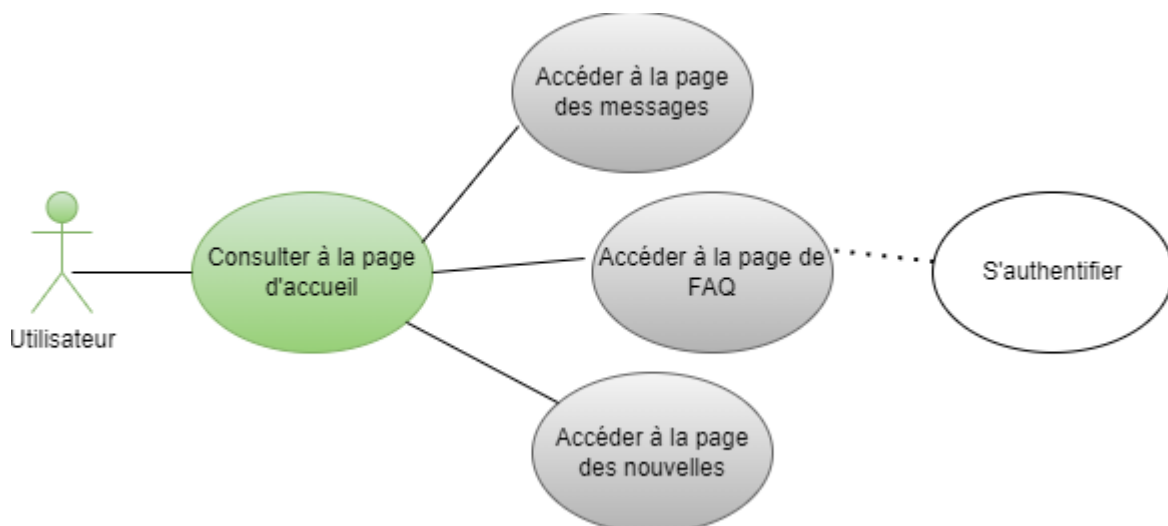


Figure 72:diagramme de cas d'utilisation sprint 5

2.2 Description textuelle du cas d'utilisation

Cas d'utilisation	Utilisateur
Acteur	Utilisateur
Pré condition	L'application web s'ouvrira directement sur l'interface d'accueil en première instance.
Post condition	L'interface d'accueil est affichée avec succès.
Scénario nominal	1. L'utilisateur accède aux interfaces d'accueil. 2. L'utilisateur peut accéder à l'interface des messages, de FAQ et de nouvelles.
Exception	E1 : Problème d'accéder à l'une des interfaces

Tableau 17: description textuelle de diagramme de cas d'utilisation sprint 5

3. Maquettage :

La maquette présente l'interface d'accueil :

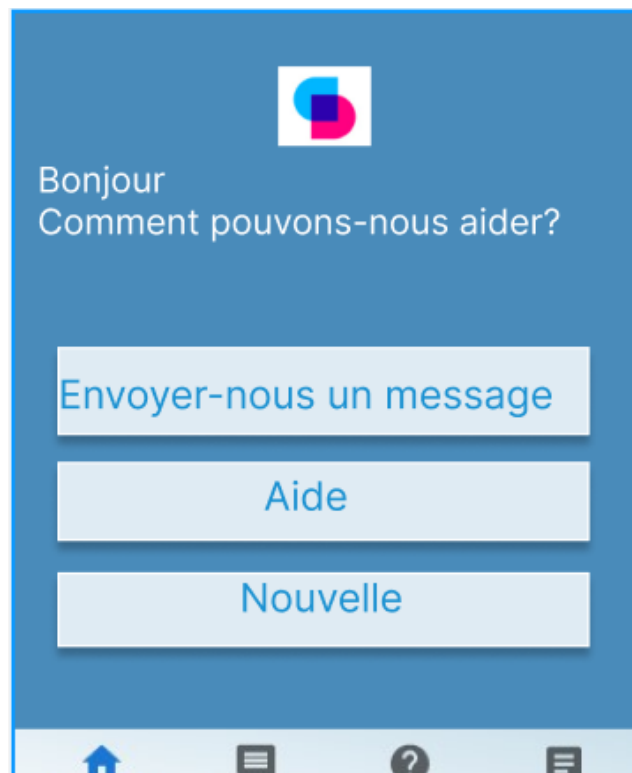


Figure 73: maquette de l'interface d'accueil

4. Conception

4.1 Diagramme de classes

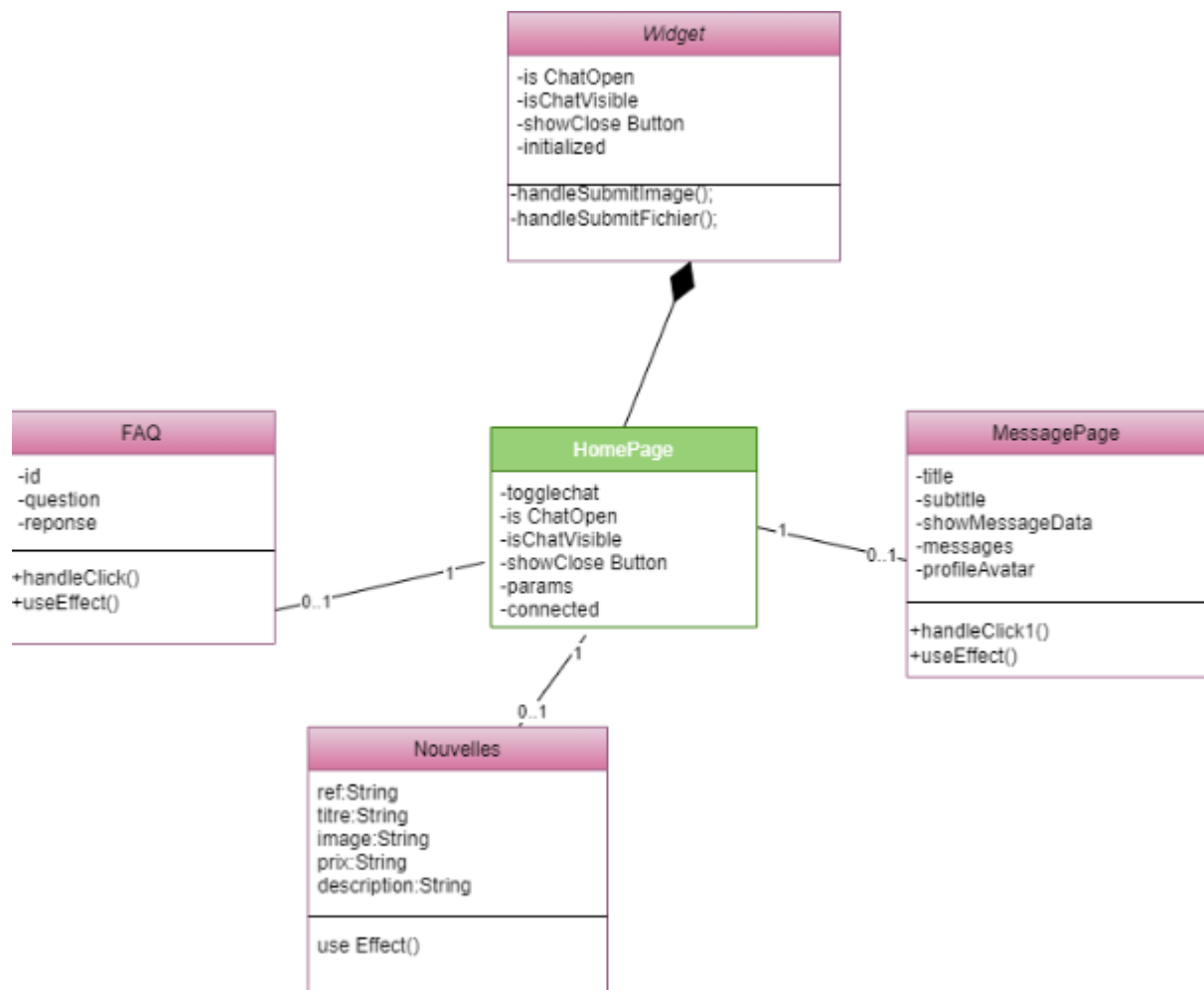
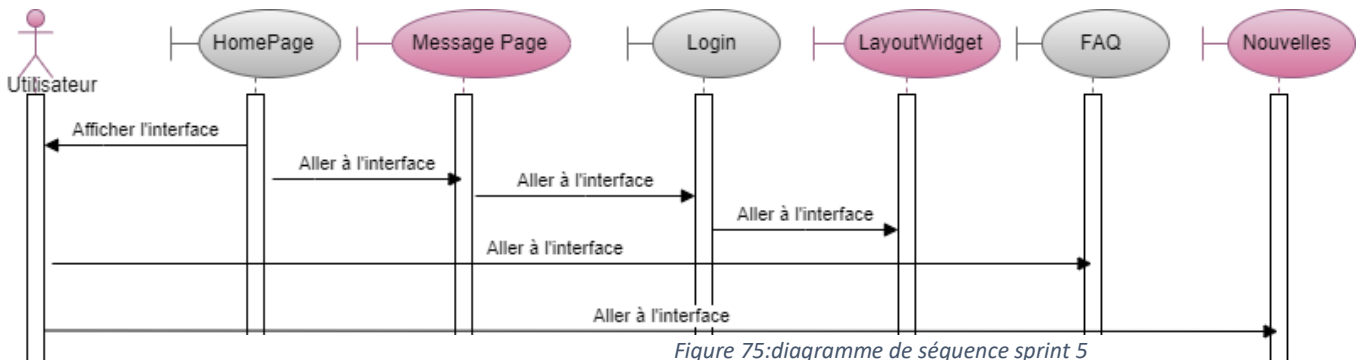


Figure 74:diagramme de classes sprint 5

4.2 Diagramme de séquence



5. Réalisation

Pendant cette phase, nous concentrons sur la création d'une interface d'accueil attrayante pour offre à l'utilisateur une vue agréable. Nous ajoutons une barre de menu en bas de cette interface, ainsi que des boutons pour facilite la navigation de l'utilisateur vers les autres interfaces.

Nous utilisons la bibliothèque de styles React « Mui-Material », pour créer le menu.

Tout d'abord, nous installons les packages nécessaires de Mui material.

```
> npm install @mui/icons-material @mui/material @emotion/styled @emotion/react
.....] \ rollbackFailedOptional: verb npm-session 00d5a357b78bd3ad
```

Figure 76:installation les packages de mui material

Ensuite, nous effectuons les importations nécessaires, telles que le package « Box » qui contient tous les boutons de navigation, et le package « Home » qui contient l'icône du menu sur la page d'accueil comme indiqué dans la capture:

```
import Box from "@mui/material/Box";
import BottomNavigation from "@mui/material/BottomNavigation";
import BottomNavigationAction from "@mui/material/BottomNavigationAction";
import { Home } from "@mui/icons-material";
import { Comment } from "@mui/icons-material";
import { Help } from "@mui/icons-material";
import { Article } from "@mui/icons-material";
```

Figure 77:les importations de package mui

Enfin, en utilisant ces packages importées, nous définissons le style de boutons et nous attribuons à l'état « value » la valeur du menu, c'est-à-dire l'interface sélectionnée par l'utilisateur. Nous associons également des icones à chaque bouton.

```

<div className="menu">
  <Box sx={{width:230}} >
    <BottomNavigation
      style={{background: "none"}}
      showLabels
      value={value}
      onChange={(event, newValue) => {
        handleChange(newValue);
      }} >
      <BottomNavigationAction label="Accueil" icon={<Home/>} />
      <BottomNavigationAction label="Messages" icon={<Comment/>} />
      <BottomNavigationAction label="Aide" icon={<Help/>} />
      <BottomNavigationAction label="Nouvelle" icon={<Article/>} />
    </BottomNavigation>
  </Box>
</div>
| | | </div>

```

Figure 78:code de menu

Voici un aperçu de l'interface :



Figure 79:interface d'accueil

Conclusion générale et perspectives

Ce projet a été réalisé dans le cadre de projet de fin d'études pendant quatre mois de stage au sein de la société Data Sphera pour obtenir le diplôme en Licence appliquée en technologie de l'informatique.

Pendant tout le stage, nous avons eu l'opportunité de mettre en pratique nos connaissances théoriques acquises lors de notre formation à l'ISET et d'enrichir nos compétences par la participation à la conception et le développement d'une application web chat.

Effectivement cette application permet :

- La possibilité d'organiser les tâches par SCRUM
- La création des interfaces avec la framework Reactjs
- L'enregistrement des fichiers dans la plateforme firebase
- L'utilisation de Redux pour afficher l'image dans la conversation et l'envoyer à 'application exactly'
- La récupération de données d'api autorisée
- Création d'une backend avec Node js et Mongo DB

Dans ce stage nous avons rencontrés des difficultés tel que :

- L'installation des packages de version incompatible
- L'authentification de l'api de l'Exactly pour récupérer la liste de FAQ
- Lorsque nous utilisons ngrok pour effectuer une requête GET, au lieu de recevoir la liste demandée, nous obtenons une page html de ngrok
- En raison de l'expiration du projet Firebase, nous sommes contraints de créer et de configurer nouveaux projet chaque mois

Ce projet répond aux besoins actuellement identifiés, mais il offre également des possibilités d'amélioration et d'optimisation en ajoutant de nouvelles fonctionnalités. Parmi les ajouts potentiels, on peut envisager l'inclusion d'une option d'envoi de vidéos, la possibilité d'envoyer des informations de localisation géographique, ainsi que la mise en place d'un système de feedback pour diffuser les commentaires des utilisateurs et améliorer l'expérience du chat web.

Néographie

[N1] [02/02/2023] <https://datasphera.com/>

[N2] [12/03/2023] <https://rasa.com/docs/rasa/>

[N3] [05/04/2023] <https://www.atlassian.com/fr/agile/scrum>

[N4] [10/04/2023] <https://socket.io/fr/docs/v4/>

[N5] [15/05/2023] <https://www.mongodb.com/fr-fr/what-is-mongodb>